



Article

An Empirical Study of the Code Generation of Safety-Critical Software Using LLMs

Mingxing Liu, Junfeng Wang, Tao Lin, Quan Ma, Zhiyang Fang and Yanqun Wu



Article

An Empirical Study of the Code Generation of Safety-Critical Software Using LLMs

Mingxing Liu ^{1,2} , Junfeng Wang ^{1,*}, Tao Lin ^{1,*}, Quan Ma ², Zhiyang Fang ¹ and Yanqun Wu ^{1,2}

¹ College of Computer Science, Sichuan University, Chengdu 610065, China; lmxstar@163.com (M.L.); fangzhiyang@scu.edu.cn (Z.F.); m15720123548@163.com (Y.W.)

² Science and Technology on Reactor System Design Technology Laboratory, Nuclear Power Institute of China, Chengdu 610041, China; maq_np@163.com

* Correspondence: wangjf@scu.edu.cn (J.W.); lintao@scu.edu.cn (T.L.)

Abstract: In the digital era of increasing software complexity, improving the development efficiency of safety-critical software is a challenging task faced by academia and industry in domains such as nuclear energy, aviation, the automotive industry, and rail transportation. Recently, people have been excited about using pre-trained large language models (LLMs) such as ChatGPT and GPT-4 to generate code. Professionals in the safety-critical software field are intrigued by the code generation capabilities of LLMs. However, there is currently a lack of systematic case studies in this area. Aiming at the need for automated code generation in safety-critical domains such as nuclear energy and the automotive industry, this paper conducts a case study on generating safety-critical software code using GPT-4 as the tool. Practical engineering cases from the industrial domain are employed. We explore different approaches, including code generation based on overall requirements, specific requirements, and augmented prompts. We propose a novel prompt engineering method called Prompt-FDC that integrates basic functional requirements, domain feature generalization, and domain constraints. This method improves code completeness from achieving 30% functions to 100% functions, increases the code comment rate to 26.3%, and yields better results in terms of code compliance, readability, and maintainability. The code generation approach based on LLMs also introduces a new software development process and V-model lifecycle for safety-critical software. Through systematic case studies, we demonstrate that, with appropriate prompt methods, LLMs can auto-generate safety-critical software code that meets practical engineering application requirements. It is foreseeable that LLMs can be applied to various engineering domains to improve software safety and development efficiency.

Keywords: generative pre-training (GPT); large language models (LLMs); safety-critical software; code generation; prompt engineering



Citation: Liu, M.; Wang, J.; Lin, T.; Ma, Q.; Fang, Z.; Wu, Y. An Empirical Study of the Code Generation of Safety-Critical Software Using LLMs. *Appl. Sci.* **2024**, *14*, 1046. <https://doi.org/10.3390/app14031046>

Academic Editor: Juan Pavón

Received: 28 December 2023

Revised: 16 January 2024

Accepted: 18 January 2024

Published: 26 January 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The software development and verification in safety-critical domains such as nuclear energy, aviation, the automotive industry, and rail transportation have been hot topics of interest in both academia and industry. Enhancing the trustworthiness and development efficiency of safety-critical software is a challenging task. Model-driven and formal methods have emerged as mainstream techniques to address this challenge [1,2]. In the industry, model-driven code generation tools such as SCADE software have been widely applied in domains such as nuclear energy, aviation, the automotive industry, and rail transportation [3–5]. However, as dedicated tools for safety software development, they also face issues such as steep learning curves, usability challenges, and high costs.

Since 2022, Generative pre-training large language models (LLMs) technology, represented by ChatGPT, has attracted attention in various fields [6]. Due to its powerful natural language understanding and content generation capabilities, LLMs technology

has great potential in code generation. Microsoft conducted detailed testing and research on code generation using LLMs, specifically GPT-4, in some general algorithms, HTML game development, and graphical user interface programming [7]. So far, code generation research based on LLMs is still in the preliminary exploration stage and lacks practical cases from industrial domains. The current research [8,9] mainly focus on the ability of code LLMs to handle basic programming elements (such as integers and strings) with general-purpose algorithms. However, industrial software containing domain-specific features is more complex than basic algorithms and requires a different approach to prompt engineering. Especially, there is no relevant research on code generation using LLMs for safety-critical software in domains such as nuclear energy and the automotive industry. For a long time, the development of safety-critical software was difficult and long-term work, which has been faced with low development efficiency, verification difficulties, a lack of experienced personnel, and other problems. The demand for new development tools is very urgent. LLMs can be significant for safety-critical development if they can significantly improve development efficiency while ensuring good quality. This paper explores the application of LLMs technology in safety-critical software development, taking into consideration the practical requirements in domains such as nuclear energy and the automotive industry.

The contributions of this study are as follows:

- A first ever study that demonstrates that GPT-based large language models can generate safety-critical software code that meets the requirements of the safety-critical industry domain.
- This study compares different approaches and methods for code generation, including overall requirement-based code generation, specific requirement-based code generation, and augmented prompt-based code generation. It proposes a new augmented prompt method called Prompt-FDC, which is suitable for generating safety-critical software code.
- This research presents a new software development process and lifecycle V model based on LLMs code generation for safety-critical software.
- This study provides two specific examples of safety-critical software from the industrial domain, which can be used by other researchers to explore further code generation methods.

2. Backgrounds and Motivation

2.1. Generative Pre-Training Large Language Models

In 2017, Vaswani et al. from Google proposed the Transformer deep learning model architecture, which laid the foundation for the current mainstream algorithm architecture in large models [10]. In 2018, Google introduced the large-scale pre-trained language model BERT (bidirectional encoder representations from transformers) based on the Transformer framework [11]. This model is a bidirectional deep pre-trained model based on Transformer, and its model parameters exceeded 300 million for the first time (BERT-Large has approximately 340 million parameters). In the same year, OpenAI introduced the generative pre-training Transformer model, GPT (generative pre-training) [12], demonstrating the potential of large models in natural language processing and driving the development of the natural language processing field. In 2020, OpenAI released the GPT-3 model [13] with approximately 175 billion parameters. GPT-3 demonstrated capabilities surpassing human-level performance in multiple natural language processing (NLP) tasks. In January 2021, Google Brain proposed the Switch Transformer model [14], becoming the first trillion-parameter language model in history, with a parameter count as high as 1.6 trillion. In December of the same year, Google also introduced GLaM [15], a general sparse model with 120 million parameters, which outperformed GPT-3 in multiple few-shot learning tasks. In March 2022, OpenAI unveiled InstructGPT (175B), which utilized instruction learning and reinforcement learning from human feedback (RLHF) to guide the model's training [16]. Using a similar model structure and training approach, and trained on a large-

scale dialogue dataset, OpenAI released ChatGPT on 30 November 2022 [17]. The language understanding ability and content generation effectiveness of ChatGPT were astonishing, and within just two months, the number of monthly active users of ChatGPT exceeded 100 million. ChatGPT marked a milestone for large language models, truly showcasing the powerful capabilities of pre-trained large models and making people fully aware of the enormous potential of pre-trained large language models. On 14 March 2023, OpenAI released a new milestone for pre-trained large language models: the more powerful GPT-4. GPT-4 not only significantly improved answer accuracy but also possessed higher-level image recognition capabilities and the ability to generate lyrics and creative text with style variations [18]. On 10 May 2023, Google officially released the new general-purpose large language model PaLM2, which supports mathematical operations, software development, language translation inference, and natural language generation [19]. On 13 July 2023, Anthropic announced the release of its latest artificial intelligence model, Claude 2, which allows inputs of up to 100,000 tokens per prompt (equivalent to approximately 750,000 English words) and generates longer documents, capable of writing thousands of tokens at once (equivalent to several thousand words or characters). Claude 2 achieved this breakthrough in terms of input and output token lengths [20]. On 19 July 2023, Meta released the open-source large language model Llama2, becoming the first freely available open-source large language model for commercial use [21].

2.2. Code LLMs

The software code can be considered as a special language. It is evident that large language models with language understanding and generation capabilities have strong potential in code generation. The previously mentioned models, GPT-3, ChatGPT, GPT-4, PaLM 2, Claude 2, and Llama2, all possess the ability to generate code. In addition to general-purpose large language models, several dedicated code models have been developed, such as DeepMind's AlphaCode [22], Salesforce's CodeGen [23], CodeT5+ [24], Meta's InCoder [25], OpenAI, and GitHub's Codex [8]. They have performed exceptionally well on popular code completion benchmarks such as HumanEval [8] and MBPP [26]. Based on the current test results, GPT-4 remains at the forefront in code generation [27].

The evaluation of code generation capabilities in large language models is an important issue. OpenAI introduced the HumanEval benchmark [8]. In HumanEval, a new evaluation set was released to measure functional correctness in synthesizing programs from docstrings. HumanEval evaluates functional correctness on a set of 164 handwritten programming problems, such as "words_string", "is_prime", "add_elements", "vowels_count", "multiply", "even_odd_palindrome", etc. The Mostly Basic Programming Problems (MBPP) dataset contains 974 programming tasks in Python [26]. The programming tasks range from simple numeric manipulations or tasks that require a basic usage of standard library functions to tasks that require nontrivial external knowledge, such as the definition of particular notable integer sequences. DS-1000 is a code generation benchmark with one thousand data science problems spanning seven Python libraries, such as NumPy and Pandas [28]. In 2023, OpenAI further proposed a new evaluation benchmark called EvalPlus. EvalPlus augments a given evaluation dataset with large amounts of test cases newly produced by an automatic test input generator, powered by both LLM- and mutation-based strategies. EvalPlus more accurately reflects the true performance of LLMs for code synthesis [9].

Microsoft extensively tested the GPT model on GPT-4 for code generation in various areas such as LeetCode algorithms, visualization of IMDb data, 2D HTML game development, and graphical user interface programming. The tests demonstrated that GPT-4 is capable of coding at a very high level [7].

2.3. Prompt Engineering for Code LLMs

From the perspective of applying large language models, prompts are the sole input for these models. It is evident that in order to achieve good generation results, the most

crucial aspect is to provide the most appropriate prompts for the large language model, which is known as prompt engineering. The quality of the output(s) generated by conversational LLMs is directly related to the quality of the prompts provided by the user [29]. In 2023, OpenAI released dedicated guidelines for prompt engineering. The importance and validation of good prompt design with LLMs is well proven [30–35]. Specifically, in the context of code generation based on large language models, Luo et al., Austin et al., Chen et al., Lai et al., Bubeck et al., and Koziol et al. have proposed prompt methods for code generation [7,8,26–28,36]. These methods primarily generate prompts from software functional descriptions, code language constraints, detailed step descriptions, and functional examples, among other aspects, as shown in Table 1. These prompt methods have achieved certain effectiveness in testing various large language models, mainly targeting general program functionalities and sub-algorithms. However, no exploration has been conducted regarding prompt engineering for top-level functional software in the industrial domain. In addition, these research studies mainly focus on Python, Java, and other high-level languages, but rarely on the generation of C language in the embedded system.

Table 1. Summary of prompts of code LLMs.

Reference	Prompt Examples	Feature
Luo et al., 2023 [27]	Prompt1: “Write a Python function to tell me what the date is today”; Prompt2: “List the prime numbers between 20 and 30 with Java”.	function descriptions and code language constraints
Austin et al., 2021 [26]	Prompt1: “Write a python function to check if a given number is one less than twice its reverse”	function descriptions and code language constraints
Chen et al., 2021 [8]	Prompt1: “Write a function vowels_count which takes a string representing a word as input and returns the number of vowels in the string. Vowels in this case are ‘a’, ‘e’, ‘i’, ‘o’, ‘u’. Here, ‘y’ is also a vowel, but only when it is at the end of the given word. Example: >>> vowels_count(“abcde”) 2 >>> vowels_count(“ACEDY”) 3”	software function descriptions and functional examples
Lai et al., 2022 [28]	Prompt1: “How to convert a numpy array of dtype=object to torch Tensor? array([array([0.5, 1.0, 2.0], dtype=float16), array([4.0, 6.0, 8.0], dtype=float16) , dtype=object)”	function descriptions
Bubeck et al., 2023 [7]	Prompt1: “Can you write a 3D game in HTML with Javascript, I want: -There are three avatars, each is a sphere. -The player controls its avatar using arrow keys to move. -The enemy avatar is trying to catch the player. -The defender avatar is trying to block the enemy. -There are also random obstacles as cubes spawned randomly at the beginning and moving randomly. The avatars cannot cross those cubes. -The player moves on a 2D plane surrounded by walls that he cannot cross. The wall should cover the boundary of the entire plane. -Add physics to the environment using cannon”.	software function descriptions, code language constraints, detailed step descriptions

Table 1. Cont.

Reference	Prompt Examples	Feature
Koziolek et al., 2023 [36]	Prompt1: Write a self-contained function block in 61131-3 structured text that implements a counter. Prompt2: Write a self-contained function block in IEC 61131-3 structured text to compute the mean and standard deviation for an input array of 100 integers.	function descriptions and code language constraints

Based on the analysis in Table 1, the prompt methods that include software function descriptions and code language constraints can be referred to as “Prompt-simple”. On the other hand, the method that combines software function descriptions, code language constraints, detailed step descriptions, and functional examples can be referred to as “Prompt-specific”.

2.4. Motivation

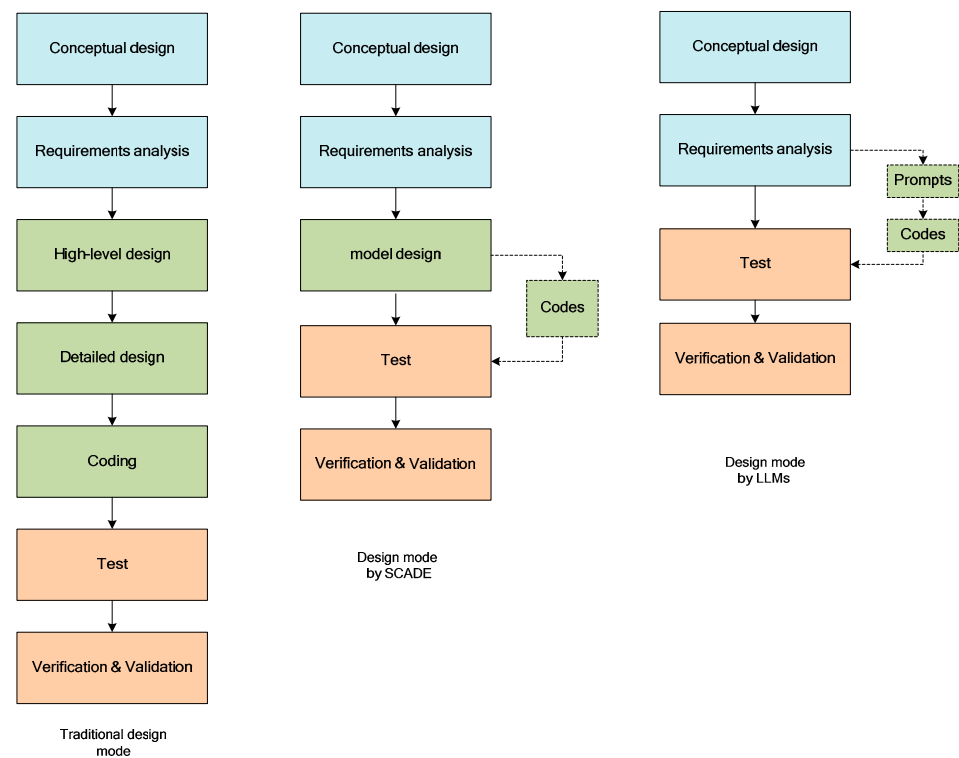
The current research on code generation based on GPT is still in the preliminary exploration stage and lacks cases from industrial domains, especially in the generation of code for safety-critical software in fields such as nuclear energy and the automotive industry. As software scales in the industrial domain continue to grow and the demand for software development efficiency increases, the support provided by large language models for code generation offers new possibilities for industrial software development. However, industrial software often contains domain-specific features, and conventional prompt engineering methods may not yield optimal results. The current benchmarks (HumanEval, MBPP, etc.) mainly focus on the ability of code LLMs to handle basic programming elements (such as integers and strings) with general-purpose algorithms. These benchmarks emphasize code generation capabilities for low-level requirements, but lack validation for high-level functional requirements. Furthermore, the current benchmarks primarily test code generation by LLMs from the perspective of multiple use cases and various algorithms, emphasizing their ability to generate code for different programming needs. However, there is a lack of research on how to refine prompts for a specific requirement to achieve the best code generation results. Therefore, this paper shifts the focus from the breadth of code generation to the depth of code generation, aiming to explore the application of LLMs in code generation for safety-critical domains by starting with specific domain requirements. This study proposes prompt engineering methods suitable for code generation in safety-critical domains.

3. Methodologies

3.1. A New Paradigm for Safety-Critical Software Development

There are typically two modes of safety-critical software development in fields such as nuclear energy and the automotive industry. The first mode is manual coding, which involves the following development processes: conceptual design, requirement analysis, high-level design, detailed design, software coding, software testing, and verification. The second mode is model-based development, with SCADE software (16.4) being a representative example. According to the traditional V-model, the main development processes in this mode include conceptual design, requirement analysis, model design, software testing, and verification. Due to SCADE’s ability to auto-generate code, the software coding phase is reduced in the development process.

Based on the principles of code generation from large language models, the starting point for code generation is software requirements. As long as there are clear prompts, a large language model can generate corresponding software code. Therefore, the safety software development method based on large language models presents a new mode, with the main development processes including conceptual design, requirement analysis, software testing, and verification. As shown in Figure 1, the development flow and stages are significantly reduced in this new mode, leading to a significant improvement in safety-critical software development efficiency.



Note: The above is a simplified version of the complete V model for safety critical software development.

Figure 1. New development modes of safety-critical software with LLMs.

In the new safety-critical software development paradigm, the main task for developers is to articulate accurate requirements and enable LLMs to better understand them.

3.2. A New Method of Code Generation for Safety-Critical Software

In this section, we describe the methodology to conduct our empirical study, which is summarized in Figure 2.

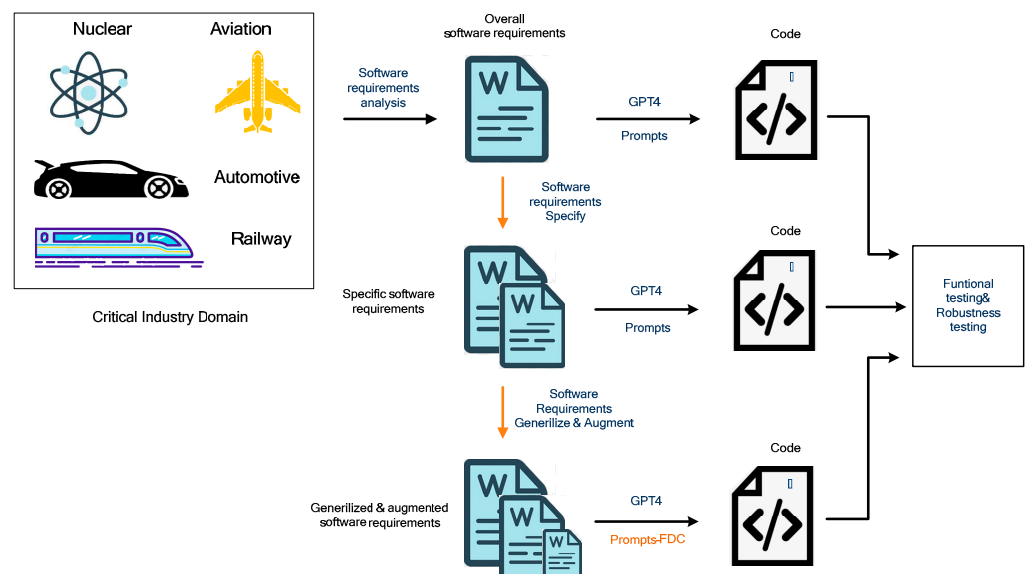


Figure 2. The methodology to conduct our empirical study.

3.2.1. Software Requirement Analysis of Safety-Critical Software

In order to conduct experiments on code generation, it is important to identify typical software requirements in safety-critical domains. First, we need to identify the domain from which the experimental examples come. Nuclear energy is a safety-critical area subject to national regulation. Its regulatory requirements for software are representative of safety-critical areas. The automotive industry is the most widely used field of software in the civil safety-critical field, and its software functions are representative. Secondly, safety-critical software mainly includes data flow control software that processes signals logically and state machine software that controls states or modes. The experimental examples must cover two types of software. Moreover, to ensure the representativeness of the research instance and facilitate the code generation experiments, the chosen instances should consist of relatively independent software modules with moderate complexity and conspicuous domain-specific features. Based on these principles, we have selected a reactor shutdown algorithm from the nuclear energy domain and cruise control software from the automotive domain. Once the software objects have been identified, requirement analysis needs to be conducted. The requirement analysis phase is based on the software development process in the nuclear energy and automotive domains. Domain engineers can conduct this requirement analysis, without the need for them to possess knowledge and expertise in code generation using LLMs. This approach minimizes the learning curves associated with applying new technologies.

3.2.2. Code Generation Based on Overall Software Requirements

After the requirements are clarified, the research moves on to the code generation phase. Given the overall requirements of the domain, a straightforward approach is to attempt code generation directly from these overall requirements. This raises the first research question addressed in this paper:

RQ1: *Can LLMs generate software code that meets the functional requirements directly from the overall requirements of safety-critical software?*

To verify this question, the overall software requirements are used as prompts and input into GPT-4. GPT-4 will auto-generate the corresponding code. The correctness of the generated code is then validated through manual code review and testing. This approach adopts the “Prompt-simple” method.

3.2.3. Code Generation Based on Specific Software Requirements

In addition to the overall requirements of the domain, specific requirements can also be used as inputs to the LLMs to generate code. Since specific requirements provide a more detailed description of software function, intuitively, we expect the quality of the generated code to be higher. However, there are doubts about whether it is possible to achieve all the functions, especially the domain-specific requirements. This leads to the second research question addressed in this paper:

RQ2: *Can LLMs generate software code that satisfies all the functions, especially the domain-specific requirements, from the specific requirements of safety-critical software?*

To verify this question, the specific software requirements need to be provided as prompts to GPT-4, which will auto-generate the corresponding code. The correctness of the generated code will then be validated through a manual code review and testing. This approach adopts the “Prompt-specific” method.

3.2.4. Code Generation Based on Software Requirements through Augmented Prompt

Based on specific requirements, we can generate code that is closer to complete requirements. However, for safety-critical domains, implemented functional requirements of software are only basic requirements. Aspects such as compliance of software to standards and safety are also crucial requirements that need to be emphasized. Only when the code

meets the industry's regulations it can be applied to practical engineering. Therefore, this raises the third question that this paper aims to investigate:

RQ3: *How can augmented prompts be provided to enable LLMs to generate code that meets all functional requirements and comply with industry regulations?*

To validate this question, it is necessary to study the augmented prompts method suitable for safety-critical software. These augmented prompts will be inputted into GPT-4, which will auto-generate the corresponding code. The correctness of the code will then be verified through a manual code review and testing. In this approach, we propose a new augmented prompt method called Prompt-FDC.

4. Experiments

4.1. Software Requirements Analysis in Nuclear and Automotive Domain

Typically, in safety-critical domains such as nuclear energy and the automotive industry, requirements engineers conduct the software requirements analysis according to industry standards. In the nuclear energy field, the relevant standards for software development include IEC 61508 [37] and IEC 60880 [38]. In the automotive field, software development standards such as ISO 26262 [39] are followed. Software requirements are usually derived from upstream system requirements and conceptual designs, which are primarily described in natural language. Requirements engineers transform the upstream natural language documents into more detailed software-specific requirements. Due to the strong natural language understanding capabilities of LLMs, in theory, it is possible to generate code from both system requirements and conceptual design documents as starting points. Clearly, the well-established requirements analysis and the requirement documents in safety-critical domains provide a solid foundation for code generation based on LLMs.

4.1.1. Requirements for Shutdown Algorithm in the Nuclear Energy Domain

In the nuclear energy domain, we investigate code generation using LLMs using an example of a shutdown algorithm involving reactor safety (referred to as the NucRate algorithm). Based on system requirements and conceptual design, the overall requirement description for this shutdown algorithm is as follows in Box 1:

Box 1. The overall requirement description for this shutdown algorithm.

The reactor protection system receives a power range neutron flux rate of positive change signals and an external reset signal from four measurement channels. The high signal of the power range neutron flux rate of positive change from the four channels is passed through a hysteresis threshold comparator to generate a signal action signal. This signal is input to the S terminal of an RS flip-flop, while the external reset signal is input to the R terminal of the flip-flop. The output signal of the RS flip-flop is input to a 4-to-2 voter, and after the 4-to-2 voting process, two shutdown signals are generated. One shutdown signal is output to a dedicated device A, while the other shutdown signal is sent to the shutdown circuit breaker. It should be noted that the RS flip-flop has a priority on the S terminal. The 4-to-2 voter has a quality bit voting degradation function. When all the quality bits of the signals are bad, the voter outputs an action signal.

After the specific requirements analysis, the itemized software requirements that have been identified are as follows in Box 2:

Box 2. The itemized software requirements for this shutdown algorithm.

1. Input signal names and sources:
 RNI_INR_01: Positive rate of change of neutron flux from four measurement channels, RNI1, RNI2, RNI3, RNI4.
 RNI_INR_02: Quality bits representing the signal quality for each channel, RNI1_Q, RNI2_Q, RNI3_Q, RNI4_Q.
 RNI_INR_03: External reset signal, RNI_RST.

2. Input signal types:
 RNI_INR_03: RNI1, RNI2, RNI3, RNI4 are analog signals of floating-point type.
 RNI_INR_04: RNI1_Q, RNI2_Q, RNI3_Q, RNI4_Q are digital signals of boolean type.
 RNI_INR_05: External reset signal, RNI_RST, is a digital signal of boolean type.

3. Valid range of input signals:
 RNI_INR_06: The upper limit for RNI1, RNI2, RNI3, RNI4 is 10000, and the lower limit is 10.

4. Output signal names and destinations:
 RNI_OUTR_01: One shutdown signal, RT1, is output to a dedicated location, A.
 RNI_OUTR_02: One shutdown signal, RT2, is sent to the shutdown circuit breaker.

5. Output signal types:
 RNI_OUTR_01: RT1 is a digital signal of boolean type.
 RNI_OUTR_02: RT2 is a digital signal of boolean type.

6. Valid range of output signals:
 None.

7. Program parameters:
 None.

8. Functional requirements:
 RNI_FUNR_01: High signals of positive rate of change of neutron flux from the four channels are first checked against the signal range. Signals that exceed the range will have their corresponding quality bits set to bad.
 RNI_FUNR_02: High signals of positive rate of change of neutron flux from the four channels are compared with a hysteresis threshold comparator to generate action signals.
 RNI_FUNR_03: The hysteresis threshold comparator is a threshold comparator with hysteresis function. The hysteresis high threshold is 100, and the hysteresis low threshold is 95.
 RNI_FUNR_04: The action signal is input to the S terminal of an RS flip-flop, and the external reset signal is input to the R terminals of the four flip-flops.
 RNI_FUNR_05: The RS flip-flop has S priority.
 RNI_FUNR_06: The output signals of the four RS flip-flops are input to a 4-to-2 voter. After the voting process, two shutdown signals are output. One shutdown signal is output to the dedicated location A, and the other shutdown signal is sent to the shutdown circuit breaker.
 RNI_FUNR_07: The 4-to-2 voter triggers an action when at least two out of the four inputs are valid. The voter has a degradation feature where, if one input's quality bit is bad, it degrades to a 3-to-2 voter, and then to a 2-to-1 voter. When all quality bits of the signals are bad, the voter outputs an action signal.

4.1.2. Requirements for Cruise Control Functions of Automobiles

In the automotive domain, we investigate code generation based on LLMs using a typical car cruise control function as an example (referred to as the CruiseControl software 1.0). This example is derived from a demo of the SCADE software (16.4). Based on system requirements and conceptual design, the overall requirement description for the cruise control function is as follows in Box 3:

Box 3. The overall requirement description for cruise control function.

The CC system connects to the car through various inputs such as enabling/disabling the CC, setting/resuming cruise speed, sensors for the accelerator and brake pedals, and car speed. It delivers outputs such as cruise speed value, throttle command, and cruise state.

The CC system's behavior is defined by the car's state and driver's actions. Initially, the CC is off when the car starts. It can be turned on by the driver and automatically turns off when the off button is pressed. The CC regulates car speed when the car speed is within the limit and the accelerator pedal is not pressed. It is disabled when the accelerator pedal is pressed or the car speed is outside the limit, and resumes to the ON state when the accelerator pedal is not pressed and the car speed is within the limit. The CC is interrupted when the brake is pressed and resumes to either the ON or STDBY states when the resume button is pressed.

When the CC is off, the car speed is controlled using the accelerator pedal. When the CC is on, the car speed is automatically regulated. The throttle command is limited to ensure comfort during automatic regulation.

The cruise speed is managed only when the CC is enabled. It can be set to the current speed, increased or decreased by a certain increment, and maintained within a specified range. The accelerator and brake pedals are detected as pressed when their values are above a certain minimum.

The CC parameters include minimum and maximum speeds, speed increment, proportional and integral gains for the regulation algorithm, maximum throttle saturation, and minimum pedal press detection.

After the specific requirements analysis, the itemized software requirements that have been identified are as follows in Box 4:

Box 4. The itemized software requirements for cruise control function.

1. Input signal names and sources:

CC_INR_01: From the cruise control switch, signal ON for open of cruise control function, signal OFF for close of cruise control function.

CC_INR_02: From the cruise control switch, signal Resume for cruise control resuming (Resume the CC after braking).

CC_INR_03: From the cruise control speed setting switch, signal Set to set the current speed as the cruise speed.

CC_INR_04: From the cruise control speed setting switch, signal QuickDecel to decrease the cruise speed.

CC_INR_05: From the cruise control speed setting switch, signal QuickAccel to increase the cruise speed.

CC_INR_06: From the throttle sensor, signal Accel for acceleration (accelerator pedal sensor).

CC_INR_07: From the brake sensor, signal Brake for braking (brake pedal sensor).

CC_INR_08: From the vehicle speed sensor, signal Speed for vehicle speed.

2. Input signal types:

RNI_INR_03: ON, OFF, Resume, Set, QuickDecel, QuickAccel are digital signals, boolean type.

RNI_INR_04: Speed, Accel, Brake are analog signals, floating-point type.

3. Valid range of input signals:

None.

4. Output signal names and destinations:

CC_OUTR_01: Cruise speed signal (CruiseSpeed), output to the instrument panel.

CC_OUTR_02: Throttle command signal (ThrottleCmd), output to the throttle.

CC_OUTR_03: Vehicle cruise state signal (CruiseState), output to the instrument panel.

5. Output signal types:

CC_OUTR_04: CruiseSpeed is an analog signal, floating-point type.

CC_OUTR_05: ThrottleCmd is a digital signal, boolean type.

CC_OUTR_06: CruiseState is a multi-state signal, including four states: OFF, ON, STDBY, INT.

Box 4. *Cont.*

6. Valid range of output signals:
None.

7. Program parameters:
SpeedMin: 30.0 Km/h;
SpeedMax: 150.0 Km/h;
SpeedInc: 2.5 Km/h;
Kp: 8.113;
Ki: 0.5;
ThrottleSatMax: 45.0 percent;
PedalsMin: 3.0 percent.

8. Functional requirement:
8.1, Cruise Control behavior
CC_HLR_CCB_01:
When the driver starts the car, the CC shall be off.
The output CruiseState should be set to OFF
CC_HLR_CCB_02:
The CC shall be set on when the driver pushes the ON button.
CC_HLR_CCB_03:
The CC shall automatically go off when the OFF button is pressed.
CC_HLR_CCB_04:
If the car speed is at the speed limit and the accelerator pedal is not pressed, the CC shall be on and regulate the car speed.
The output CruiseState shall be set to ON.
CC_HLR_CCB_05:
The CC system shall be automatically disabled when the accelerator pedal is pressed, or the car speed is outside the speed limit.
The output CruiseState shall be set to STDBY.
CC_HLR_CCB_06:
The system shall return to the ON state when both the accelerator pedal is not pressed, and the car speed is inside the speed limit.
CC_HLR_CCB_07:
The CC shall be immediately interrupted when the brake is pressed.
The output CruiseState shall be set to INT.
CC_HLR_CCB_08:
The system shall resume either to the ON or STDBY states, depending on the Accelerator pedal and the Speed value when the resume button is pressed. The last set cruise speed shall be re-used.

8.2, Car driving control
CC_HLR_CDC_01:
When the CC is off, the car speed shall be driven using the accelerator pedal.
CC_HLR_CDC_02:
When the CC is on, the car speed shall be automatically regulated.
CC_HLR_CDC_03:
The regulation shall be performed using a proportional and integral algorithm, with Kp and Ki factors.
CC_HLR_CDC_04:
The regulation algorithm shall be protected against the overshoot of its integral part: the integral action shall be reset when the CC is going on, and frozen when the throttle output is saturated.
CC_HLR_CDC_05:
The throttle command shall be saturated at ThrottleSatMax when automatically regulating, in order to limit the car acceleration for the comfort.

8.3, Cruise Speed management
CC_HLR_CSM_01:
The Cruise Speed shall be managed only when the cruise control is enabled, meaning ON, STDBY, or INT states.
CC_HLR_CSM_02:
The Cruise Speed shall be set to the current speed when the set button is pressed.
CC_HLR_CSM_03:
The Cruise Speed shall be increased by SpeedInc km/h when the QUICKACCEL button is pressed. Only if this new value of the Cruise Speed is still lower than the maximal speed SpeedMax km/h.

Box 4. *Cont.*

CC_HLR_CSM_04:

The Cruise Speed shall be decreased with SpeedInc km/h when the QUICKDECEL button is pressed.

CC_HLR_CSM_05:

The Cruise Speed shall be maintained between SpeedMin and SpeedMax km/h values.

8.4, Pedals pressed detection

CC_HLR_PPD_01:

The accelerator pedal shall be detected as pressed when its value is above PedalsMin.

CC_HLR_PPD_02:

The Brake pedal shall be detected as pressed when its value is above PedalsMin.

4.2. Code Generation Based on Overall Software Requirements

The code generation method based on overall requirements is to use the overall software requirements as the prompt for the LLMs. Then, the LLMs auto-generate the code.

4.2.1. Code Generation Based on the NucRate Algorithm's Overall Requirements

Using the Prompt-simple approach, based on the NucRate algorithm's overall requirements, along with code language constraints, the prompt is as follows in Box 5:

Box 5. The prompt based on the NucRate Algorithm's overall requirements.

Generate the C code according to the following requirements:

The reactor protection system receives power range neutron flux rate of positive change signals and an external reset signal from four measurement channels. The high signal of the power range neutron flux rate of positive change from the four channels is passed through a hysteresis threshold comparator to generate a signal action signal. This signal is input to the S terminal of an RS flip-flop, while the external reset signal is input to the R terminal of the flip-flop. The output signal of the RS flip-flop is input to a 4-to-2 voter, and after the 4-to-2 voting process, two shutdown signals are generated. One shutdown signal is output to a dedicated device A, while the other shutdown signal is sent to the shutdown circuit breaker. It should be noted that the RS flip-flop has a priority on the S terminal. The 4-to-2 voter has a quality bit voting degradation function

Based on this prompt, GPT-4 generates the C code. The entire code is in Appendix A Listing A1.

We analyze the generated code in relation to the overall requirements: the code defines two struct types: RSFlipFlop and VoterInput. The RSFlipFlop struct represents an RS flip-flop and contains a powerSignal and a resetSignal, which represent the set input signal and the reset signal of the flip-flop, respectively. The VoterInput struct represents the input signals of a voter and includes a qualityBit and a stopSignal, representing the quality of the signal and the validity signal of the channel, respectively. The program also defines two constants: CHANNELS and THRESHOLD, representing the number of channels and the threshold for the power signal, respectively. These constants are defined using macro definitions (#define) for convenient usage throughout the program. The program includes three functions: processPowerSignal, processResetSignal, and processVoterInputs. The processPowerSignal function handles a high signal rate in a channel and sets the powerSignal of the RS flip-flop to 1 if the signal strength is greater than the THRESHOLD. It also sets the stopSignal of that channel to the powerSignal of the RS flip-flop. The processResetSignal function handles the reset signal and sets the resetSignal of the RS flip-flop to the signal parameter. The processVoterInputs function handles the input signals of the voter, counts the number of good and bad input signals, and outputs the corresponding stop signal or action signal based on their quantities. The main function of the program initializes the input signals of the RS flip-flop and the voter and simulates the power signals of CHANNELS channels. It then calls the processResetSignal function to handle the reset signal and finally calls the processVoterInputs function to handle the input signals of the voter and output the corresponding results.

Although the program is syntactically correct and can be directly executed, upon deeper analysis, the following issues can be identified:

- (1) The requirements of the threshold comparator are not understood, resulting in a missing function.
- (2) The function of the RS flip-flop is not fully implemented.
- (3) The degradation function of the quality bit in the 4-to-2 voter is still incomplete.
- (4) The entire algorithm does not form a top-level function.
- (5) The code lacks sufficient comments.

In summary, the generated code does not fully meet the software requirements and cannot be used directly.

4.2.2. Code Generation Based on the CruiseControl's Overall Requirements

Using the Prompt-simple approach, based on the CruiseControl's overall requirements, along with code language constraints, the prompt is as follows in Box 6:

Box 6. The prompt based on the CruiseControl's overall requirements.

Generate the C code according to the following requirements:

The CC system connects to the car through various inputs such as enabling/disabling the CC, setting/resuming cruise speed, sensors for the accelerator and brake pedals, and car speed. It delivers outputs such as cruise speed value, throttle command, and cruise state.

The CC system's behavior is defined by the car's state and driver's actions. Initially, the CC is off when the car starts. It can be turned on by the driver and automatically turns off when the off button is pressed. The CC regulates car speed when the car speed is within the limit and the accelerator pedal is not pressed. It is disabled when the accelerator pedal is pressed or the car speed is outside the limit and resumes to the ON state when the accelerator pedal is not pressed and the car speed is within the limit. The CC is interrupted when the brake is pressed and resumes to either the ON or STDBY states when the resume button is pressed. When the CC is off, the car speed is controlled using the accelerator pedal. When the CC is on, the car speed is automatically regulated. The throttle command is limited to ensure comfort during automatic regulation. The cruise speed is managed only when the CC is enabled. It can be set to the current speed, increased or decreased by a certain increment, and maintained within a specified range. The accelerator and brake pedals are detected as pressed when their values are above a certain minimum. The CC parameters include minimum and maximum speeds, speed increment, proportional and integral gains for the regulation algorithm, maximum throttle saturation, and minimum pedal press detection.

Based on this prompt, GPT-4 generates the C code. The entire code is in Appendix B Listing A4.

We analyzed the generated code in relation to the overall requirements: the program defines several constants, including minimum speed, maximum speed, speed increment, maximum throttle saturation, and minimum pedal pressure detection CC parameters. The program also defines three enumeration types: CCState, which represents the state of the CC system. The program defines several variables, including cruise speed, vehicle speed, throttle command, CC state, throttle pedal, and brake pedal input signals. The function `updateCCState()` in the program is used to update the CC state. The function uses if-else statements to determine the next state based on the current state and input signals. In the OFF state, if the vehicle speed is between the minimum and maximum speeds and the throttle pedal is not pressed, the program enters the ON state and sets the cruise speed to the current vehicle speed. In the ON state, if the throttle pedal is pressed, the vehicle speed is below the minimum speed or above the maximum speed, and the program enters the STDBY state. If the brake pedal is pressed, the program enters the OFF state. In the STDBY state, if the throttle pedal is not pressed and the vehicle speed is between the minimum and maximum speeds, the program enters the ON state. The function `updateThrottleCommand()` in the program is used to update the throttle command. In the ON state, a throttle command is calculated based on the difference between the current vehicle speed and the cruise speed, and it is limited by the maximum throttle saturation. In

other states, the throttle command is set to 0. The main function of the program simulates a scenario where the vehicle speed is 50, and neither the throttle pedal nor the brake pedal is pressed. It updates the CC state and throttle command and outputs the CC state, cruise speed, and throttle command. This program is just a simple simulation, and in actual applications, it would require specific hardware and operating system implementations. It would also need to continuously read input signals and update the CC state and throttle command based on those signals.

The program is syntactically correct and can be directly executed. From the program structure, it appears to fulfill the basic requirements outlined in the overall requirements. However, upon deeper analysis, the following issues can be identified:

- (1) The control function for cruise speed is not implemented.
- (2) The overall requirements do not provide detailed instructions on the control method for the throttle. LLMs have completed this part based on their own understanding, but the function is incomplete.
- (3) The entire function has not been encapsulated in a top-level function.
- (4) The code lacks sufficient comments.

Based on the analysis of the generated code in these two examples, we can conclude that LLMs usually cannot produce directly usable code based solely on the overall requirements.

4.3. Code Generation Based on Specific Requirements

4.3.1. Code Generation Based on NucRate Algorithm's Specific Requirements

By adopting a Prompt-specific approach, we combine software functional descriptions, code language constraints, and detailed step descriptions to form a new prompt. Based on this prompt, GPT-4 generates the NucRate algorithm's C code (the entire code is in Appendix A Listing A2).

Compared to the code generated based on the overall requirements in Section 4.2.1, this code undoubtedly achieves a higher level of function. The function of the threshold comparator, RS flip-flop, and 4-to-2 voter have all been implemented. However, there are still some issues:

- (1) The threshold comparator only implements the upper and lower limit function and does not include hysteresis.
- (2) The quality bit function of the 4-to-2 voter is incomplete.
- (3) The code has a low comment rate.

In summary, the generated code still does not fully meet the software requirements and cannot be used directly.

4.3.2. Code Generation for CruiseControl's Specific Requirements

By adopting a Prompt-specific approach, we combine software functional descriptions, code language constraints, and detailed step descriptions to form a new prompt. Based on this prompt, GPT-4 generates CruiseControl's C code (the entire code is in Appendix B Listing A5).

Compared to the code generated based on the overall requirements in Section 4.2.2, undoubtedly, this code achieves a higher level of function completion. The cruise speed control function has been implemented, forming a top-level function. However, there are still some issues that need to be addressed:

- (1) The throttle control implementation is incomplete.
- (2) The Pedals pressed detection function and Car driving control function are implemented within the same function, resulting in bad readability.
- (3) The car_driving_control function contains unused variables: Kp and Ki.
- (4) The input structure in the Main function is not initialized.
- (5) The code lacks sufficient comments.
- (6) It does not fully comply with industry standards (MISRA).

In summary, the code generated based on specific requirements has implemented the basic software requirements. However, further work is needed to meet the requirements for an engineering application.

4.4. Code Generation Based on Software Requirements through Augmented Prompt

Based on the practical results presented in Sections 4.2 and 4.3, the GPT model demonstrates good capability in understanding requirements and generating code that meets certain criteria. However, when it comes to understanding specific industry requirements, such as hysteresis comparators or quality bit degradation functions in voting logic, the model fails to grasp the deeper understanding needed to generate corresponding code. Basic prompts only generate code that meets basic requirements and improvements are needed for code standards, usability, maintainability, and performance. Therefore, this section attempts to enhance the quality of code generated by LLMs using more informative prompts.

4.4.1. Refinement and Generalization of Functional Requirements

For specific industry requirements, it is possible to further refine the function by focusing on specific functional points. For example, in the NucRate algorithm, the degradation of the voting quality bit function can be further refined as follows in Box 7:

Box 7. Refinement and generalization of functional requirements.

RNI_FUNR_07_01: The 4-out-of-2 voting logic takes action when at least two out of the four inputs are valid. However, the voting result is influenced by the quality bits of the inputs.
 RNI_FUNR_07_02: The degradation of the voting quality bit function for the 4-out-of-2 voting logic refers to the following: when one input has a bad quality bit, it degrades to a 3-out-of-2 voting logic; when two inputs have bad quality bits, it degrades to a 2-out-of-1 voting logic; when three inputs have bad quality bits, it degrades to a 1-out-of-1 voting logic; when all signals have bad quality bits, the voting logic directly outputs the action signal.

4.4.2. Standard Requirements for the Code

In the safety-critical domain, there are usually requirements for the code's compliance with certain programming standards and the code comment rate. These requirements can be directly provided as prompts to LLMs. For example in Box 8:

Box 8. Standard requirements for the code.

RNI_NFUNR_01: Programming language constraint: C99 standard language.
 RNI_NFUNR_02: Code standard limitation: compliance with MISRA-C: 2004 standard.
 RNI_NFUNR_03: Code comment rate: greater than 20%.

4.4.3. Structural Requirements for the Code

To improve code readability and maintainability, certain functionalities are often implemented as functions or pre-defined macros in manual coding, thereby giving the entire program a well-structured design. Such requirements can also be submitted to LLMs as augmented prompts. For example in Box 9:

Box 9. Structural requirements for the code.

RNI_NFUNR_04: Program structure requirement (readability): Implement the hysteresis threshold comparator, 4-out-of-2 voting logic, and RS flip-flop as separate functions.

4.4.4. Safety and Security Requirements for the Code

In safety-critical domains such as nuclear energy, software safety and security are of utmost importance as they directly affect the safe operation of the system. Therefore, constraints on code safety and security must be specified. However, to ensure LLMs'

understanding, specific code safety and security requirements need to be decomposed. For example, in the nuclear energy domain, it is usually required that the program's memory must be statically allocated and the use of interrupts is restricted. The aforementioned safety and security requirements can be provided to LLMs as augmented prompts. For example in Box 10:

Box 10. Safety and security requirements for the code.

RNI_NFUNR_05: Safety and security requirement: static memory allocation.
RNI_NFUNR_06: Safety and security requirement: no use of interrupts.

4.4.5. Performance Requirements for the Code

In safety-critical domains, real-time performance is a key metric for software, and thus, there is a high demand for software performance. However, since LLMs, as a large language model, cannot execute programs on their own, they cannot directly judge whether the generated code meets the time performance requirements. As a workaround, we can use performance prompts to indicate LLMs to optimize the program's performance. If the tests show that the generated code does not meet the performance requirements, we can request LLMs to perform performance analysis on the code, identify performance bottlenecks, and then optimize the performance. This process involves iterations. For example in Box 11:

Box 11. Performance requirements for the code.

RNI_NFUNR_07: Performance requirements: execution time less than 100 us.

Based on the aforementioned augmented prompting method, the NucRate algorithm and cruise control software code are re-generated (the entire code is in Appendix A Listing A3 and Appendix B Listing A6).

It is worth noting that even if we provide LLMs with all the necessary requirements, they may still miss some requirements during code generation (such as the MISRA specification mentioned earlier). In such cases, the non-implemented requirements will be reinforced by appending additional prompts. For example, if the code comment rate is insufficient, or the code does not comply with the MISRA specification, these specific requirements will be emphasized, and LLMs will generate code that fully satisfies the requirements. So, the iterations in LLMs are usually needed.

Robust testing is about improving reliability and finding those corner cases by inputting data that mimic extreme environmental conditions to help determine whether or not the system is robust enough to deliver [40]. In order to ensure its reliability and safety of the generated code, we conducted the robustness testing of the generated code. Test cases and results are shown in Table 2. The test code is in Appendix C Listing A7.

The code generated through the aforementioned augmented prompts has been manually reviewed, analyzed, and tested to ensure that it meets the corresponding functional requirements and industry standards. It can be integrated and applied in practical engineering.

Table 2. Robustness testing of generated code.

Robustness Testing	Test Cases	Results
Boundary Value Testing	Test inputs close to the boundary values or beyond the boundary values for each element in the inputs array. For example, test values near LOWER_LIMIT and UPPER_LIMIT, as well as values below LOWER_LIMIT-1 and above UPPER_LIMIT+1. Test boundary value scenarios for HYSTERESIS_HIGH_THRESHOLD and HYSTERESIS_LOW_THRESHOLD. For example, test values close to these thresholds and values beyond these thresholds.	PASS
Exception Input Testing	Test invalid data in the inputs array, such as non-numeric values or strings. Test inputs with formatting errors, such as negative values or values represented in scientific notation. Test inputs that are out of range, such as values below LOWER_LIMIT or above UPPER_LIMIT.	PASS
Random Input Testing	Use randomly generated inputs within a reasonable range to test the system's robustness under different input conditions.	PASS
Forced Error Testing	Simulate changes in the reset signal and observe if the system responds correctly and resets the corresponding outputs.	PASS
Security Testing	Test malicious data in the inputs and quality_bits arrays, such as extreme values outside the normal range or unexpected data. Simulate errors in the quality_bits array, such as setting quality bits to values that do not match the corresponding inputs, to test the system's ability to handle exceptional data securely.	PASS

5. Results

In this section, we present the results of our experiments for each of the research questions.

5.1. RQ1: Can LLMs Generate Software Code That Meets Functional Requirements Directly from the Overall Requirements of Safety-Critical Software?

According to the experimental results in Section 4.2, the answer to this question is negative. LLMs only create basic code frameworks when directly generating code for safety-critical software from overall requirements. The code does not meet the functional requirements. However, generating code from the overall requirements still has its significance. It can provide software developers with a starting point for structuring the overall framework of the code during the conceptual stage of software development. This approach is suitable for scenarios where the LLMs are used as a software development assistant tool.

5.2. RQ2: Can LLMs Generate Software Code That Meets All Functional Requirements, Especially Domain-Specific Feature Requirements, from the Specific Requirements of Safety-Critical Software?

According to the experimental results in Section 4.3, the answer to this question is partially negative. The current language understanding and generation capabilities of the LLMs are not sufficient to generate complete functional code from highly specialized domain requirements. However, there has been good progress in generating code for general control flow and input–output relationships. This generated code can directly provide development support to software developers.

5.3. RQ3: How Can Augmented Prompts Be Provided to Enable LLMs to Generate Code That Meets All Functional Requirements and Complies with Industry Regulations?

According to the experimental results in Section 4.4, the quality of generated code has significantly improved with augmented prompts. From the perspective of prompt enhancement, there are mainly two aspects. First, domain-specific functional requirements

are transformed into more general descriptions that are easier for LLMs to understand. Second, domain-specific standard requirements are extracted and transformed into non-functional constraints for code generation, thereby ensuring that the generated code meets higher domain requirements. Moreover, this augmented prompt information can usually be extracted from the requirement documents of safety-critical software. The detail procedures for conducting the experiments is shown in Appendix E Listing A10.

6. Discussions

6.1. Code Generation Comparison between LLMs and SCADE

We compare the code generation process of the proposed LLMs approach with the widely used SCADE software (16.4) development method. The two approaches differ significantly in the design input, development time, code size, learning curve, and verification method, as shown in Table 3.

Table 3. Code generation comparison between LLMs and SCADE.

Comparison Item	LLMs Code Generation	SCADE Code Generation
Input	Software Requirements	Software Models
Development Time Investment	1 h	24 h
Code Size	100+	1000+
Learning Curves	Almost no learning curve required	Long-term specialized learning
Verification	Comprehensive verification is required	Model correctness implies code correctness; model needs to be verified carefully
Domain Compliance	Complies with standards	Complies with standards

Note: cruise control software is an example.

Clearly, the language model-based approach for developing safety-critical software has distinct advantages in terms of learning curves and software development efficiency. With further improvements in code quality, this approach will provide a new choice for the development of safety-critical software.

6.2. Prompt Engineering for Code Generation of Safety-Critical Software

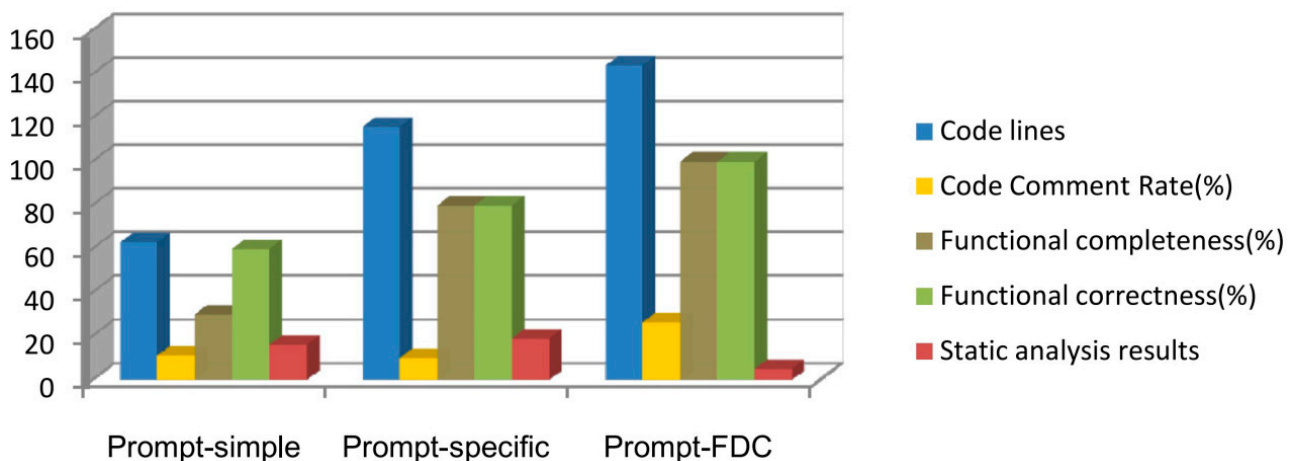
From the perspective of code generation by LLMs, the above experiments indicate that the most crucial aspect is to provide the most appropriate prompts for the models. This paper conducts experiments combining two mainstream prompt methods and proposes a new prompt method. The three prompt methods yield different effects, as shown in Table 4.

By comparison, the augmented prompt achieves the best domain code generation results. The generated code size increases from 63 lines to 144 lines, the completeness of the generated code function improves from 30% to 100%, and the correctness of the generated code function improves from partially correct to completely correct. The code comment rate increases to 26.3%, and the static analysis errors and warnings decrease from 19 to 5 warnings. Furthermore, better results are achieved in terms of code compliance, code readability, and code usability. The intuitive comparison results are shown in Figure 3. This prompt engineering framework integrates basic functional requirements, domain feature generalization, and standard specification constraints. We refer to it as Prompt-FDC.

Table 4. Code generation Comparison between different Prompts.

Comparison Item	Prompt-Simple [27,28]	Prompt-Specific [7]	Prompt-FDC (Proposed in This Paper)
Input	Prompt of overall requirements	Prompt of specific requirements	Prompt of augmented requirements
Code size	63	116	144
Code comment rate	11.1%	10%	26.3%
Completeness of generated code function	30%, function missing	80%, domain function missing	100%, complete function
Correctness of generated code function	60%, partially correct	80%, partially correct	100%, completely correct
Static analysis results *	Style warnings: 16	Errors: 4 Style warnings: 15	Style warnings: 5
Code compliance	Non-compliant	Non-compliant	Compliant with MISRA-C:2004 standard
Code readability	Moderate	Moderate	High
Code usability	Not directly usable	Not directly usable	Directly usable

* Cruise control software is an example. Static analysis results are checked using Cppcheck (2.11) [41].

**Figure 3.** Code generation comparison between different prompts.

Different from other prompt methods, Prompt-FDC starts with the requirements of safety-critical software. This well-established development practice can also be applied to code generation based on large language models. This paper does not provide a detailed description of the requirement analysis process but starts with the requirement document and proposes the new prompt engineering framework, Prompt-FDC.

As shown in Figure 4, the Prompt-FDC method can be divided into five phases, from the requirement document to prompts.

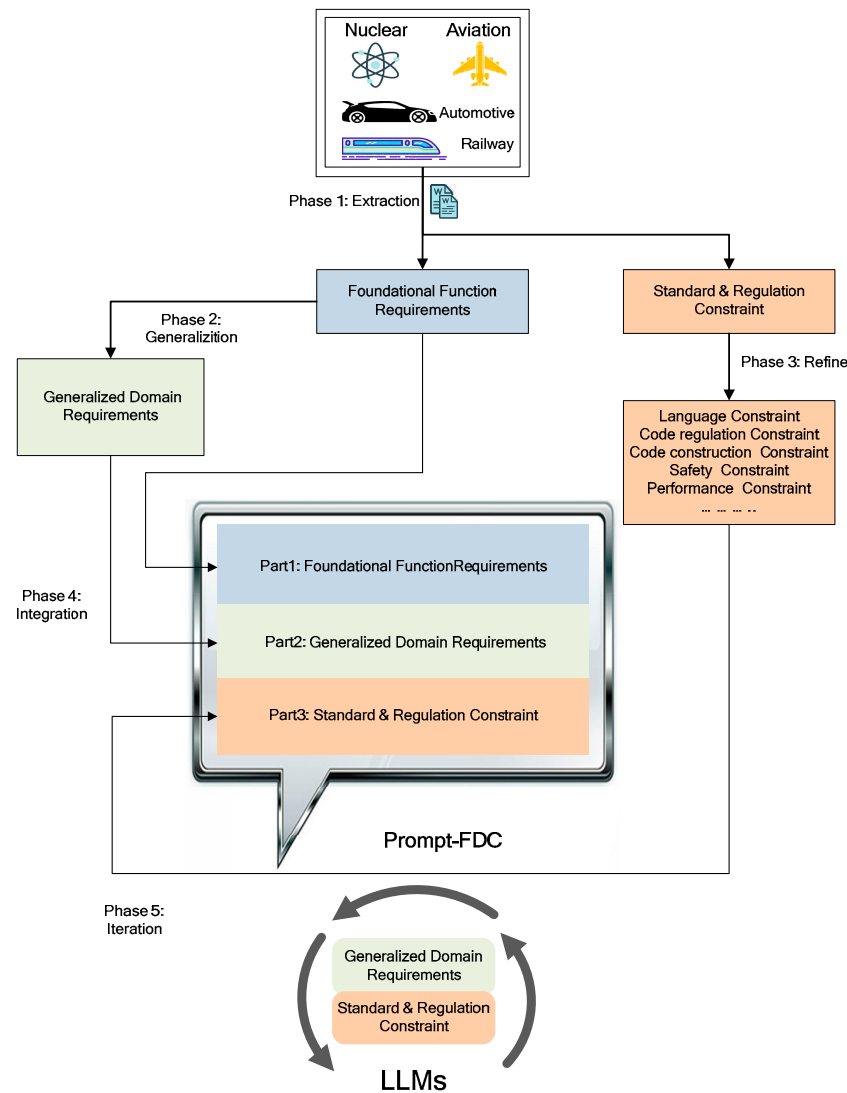


Figure 4. Prompt-FDC: prompts that integrate basic function, generalized domain requirements, and domain regulations constraints.

Phase 1: Extraction. Basic software function and domain standard specification requirements are extracted from the software requirement document.

Phase 2: Generalization. Due to the presence of numerous domain-specific terms in the description of basic software function requirements, LLMs have a weak understanding of such terms. Therefore, it is necessary to convert domain-specific terms into general language descriptions.

Phase 3: Refine. The domain standard specification requirements in the requirement document are usually top-level requirements. Specific requirements related to code implementation need to be further refined, including code standards, program structure, safety and security, performance, and other constraints.

Phase 4: Integration. The prompts from the previous three phases are integrated to form a specific and complete contextual prompt. This prompt can be directly provided to the LLMs. The final Prompt-FDC consists of three parts: part 1, the basic functional requirement part, mainly describes the overall framework, functional points, and input/output requirements of the software in a structured manner; part 2, the domain feature generalization part, describes the domain-specific functional points in a generalized manner, enabling the LLMs to understand them directly; part 3, the domain standard specification constraint part, extracts key constraints from the perspective of the domain industry standard, in-

cluding language constraints, code standard constraints, structural constraints, safety and security constraints, performance constraints, etc.

Phase 5: Iteration. This phase is optional. For complex requirements, the LLMs may not be able to generate code that satisfies all requirements at once. Therefore, based on the review and analysis of the generated code, iterative reinforcement can be performed in terms of domain requirements, specification constraints, etc. After several iterations, we can obtain higher-quality software code that meets the requirements for practical engineering applications.

The quality of code generation is heavily dependent on the skill and methods used to transform and generalize the requirements. We further standardize the requirements transformation process, shown in Figure 5. We break down the transformation process into 10 steps. Each step completes the extraction and standardization of a class of requirements. Each type of requirement is standardized into XML format. The final result is a standardized document in XML format for the FDC requirements. Adherence to this XML format for all requirements transformations ensures consistency for generalized requirements.

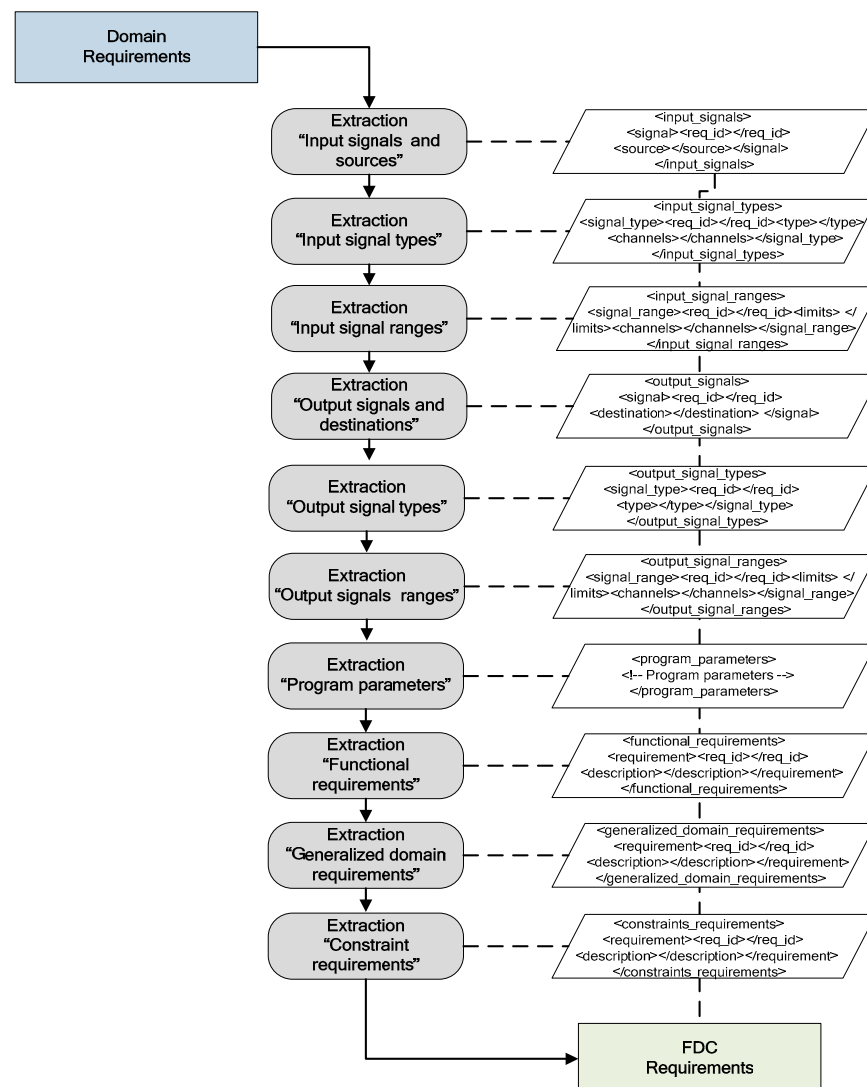


Figure 5. Standardization of requirement transformation process.

The standardized XML template for generalized requirements is shown in Appendix D Listing A8. The standardized XML generalized requirements of the NucRate algorithm are shown in Appendix D Listing A9.

As shown in Table 5, compared to other existing prompt methods discussed in Section 2.3, the proposed Prompt-FDC in this paper presents the most comprehensive prompt framework. It covers the requirements of safety-critical software and introduces a method to progressively refine and generalize domain requirements, enabling a more thorough understanding of the domain requirements and generating higher-quality code.

Table 5. Comparison between different prompts of code generation.

Prompt	Prompt-Simple		Prompt-Specific		Prompt-FDC
Reference	Luo et al., 2023 [27]	Koziolek et al., 2023 [36]	Chen et al., 2021 [8]	Bubeck et al., 2023 [7]	methodology proposed in this paper
Features	Function description, code language constraints	Function description, code language constraints	Function description, function example	Detailed step descriptions, code language constraints	Part1, function description, itemized requirements; Part 2, domain feature generalization part; Part3, domain specification constraint part

6.3. The Software Development Process Based on LLMs

From the perspective of traditional software engineering and based on the previous empirical results, we can summarize a set of code generation methods and process models using LLMs applicable to safety-critical software. This is illustrated in Figure 6. Since the requirements for safety-critical software are usually analyzed and documented according to industry standards, they can serve as the starting point for code generation. However, some domain-specific terms and functional requirements are often not fully understood by LLMs, so it is necessary to deepen the original requirements and transform domain-specific requirements into general requirements. This is referred to as requirement refinement for LLMs. Requirement refinement involves both functional and non-functional requirements. After requirement refinement, the resulting additional requirement prompts are directly provided to LLMs as input, which then generate the corresponding code automatically. After code generation, it is essential to verify the correctness of the code. We can complete this using relatively traditional methods, including code review and testing. Verification and validation, which is crucial for safety software, also needs to be performed. The entire development process is iterative. Where the testing and validation in the backend do not pass, it is necessary to trace back to the initial domain requirements and start the next iteration process.

6.4. The Software V-Model Based on Large Language Models

From the perspective of software development models, the LLMs-based safety-critical software development has formed a new V-model, as shown in Figure 7. In the new V-model, the development and validation stages have been reduced from the original 8 stages to 5 stages. In the development stage, the system design and software design stages have been eliminated, and the process directly proceeds from the requirements stage to the LLMs code generation stage. Due to the length limitation of prompt inputs for LLMs and the difficulty of accurately understanding lengthy requirements, it is recommended to generate code for individual unit functionalities rather than generating code for the whole software function. After the generation of unit function code, the next stage is the integration of unit function code to build the overall software function, which can also be facilitated by the LLMs-generated code framework. In terms of testing and validation, adjustments have been made in conjunction with code generation during development, dividing it into two stages: component testing and acceptance testing. In the component testing stage, static and dynamic testing of the unit function code generated by LLMs are performed, along with completing the corresponding functional testing based on the

requirements. In the acceptance testing stage, system testing and acceptance testing are conducted specifically for the overall software requirements.

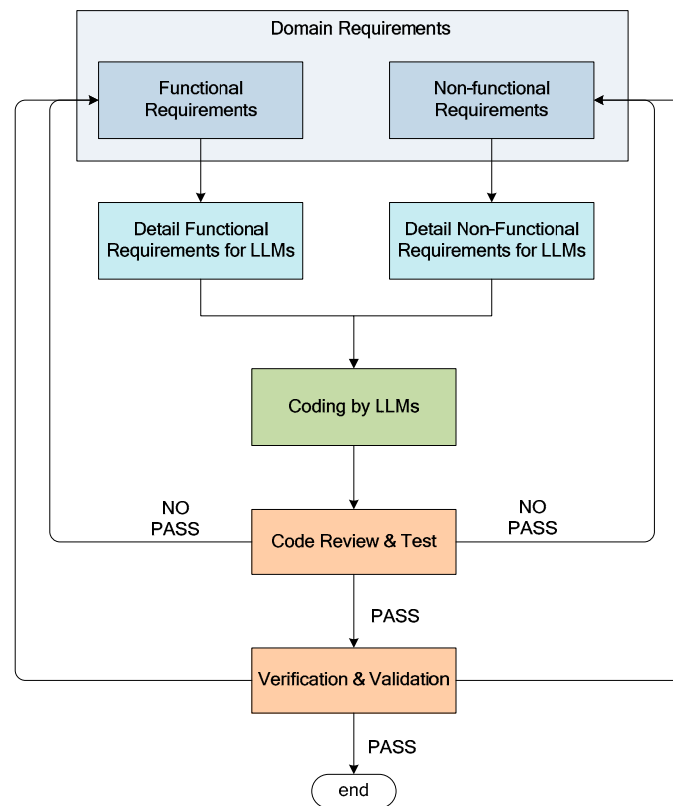
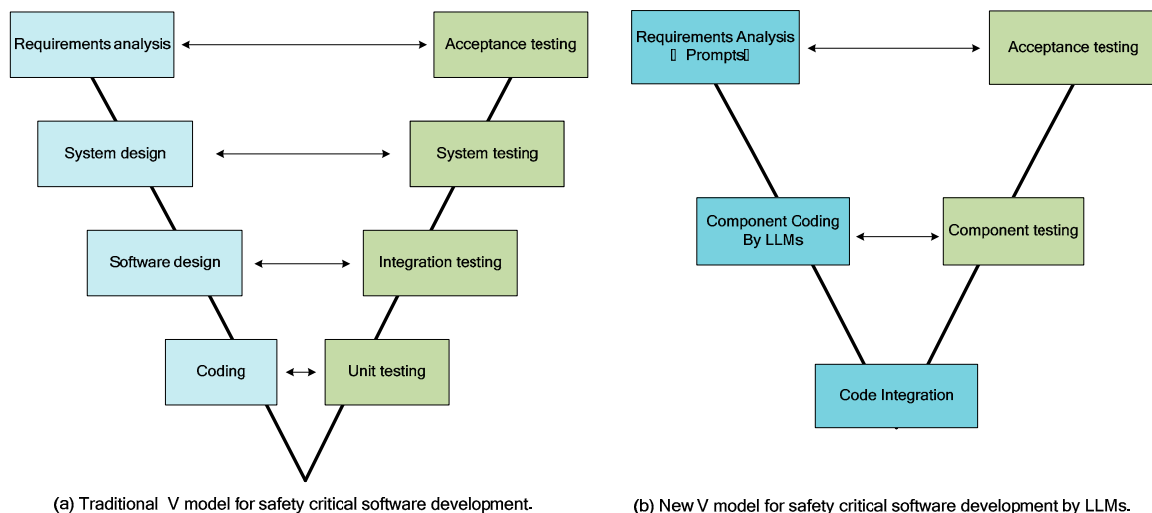


Figure 6. The development process of safety-critical software by code LLMs.



(a) Traditional V model for safety critical software development.

(b) New V model for safety critical software development by LLMs.

Figure 7. The new V-model of safety-critical software by code LLMs.

7. Threats to Validity

In this section, we describe the limitations of our paper:

Internal Validity: The degree of generalization and concretization of domain requirements will directly impact the final code quality. The goal of this transformation is to provide the LLMs with the most general functional and non-functional constraint descriptions. In the experiments conducted in this paper, the requirement transformation for the

quality degradation function of the 4-out-of-2 voter has undergone multiple iterations, and the final requirement description has become the most general representation.

External Validity: The code generation model used in this paper is GPT-4, which is also undergoing iterative optimization. Therefore, using the code generation prompts provided in this paper may result in code that differs from what we present in this paper, even in terms of the implementation approach and style. As for code generated by other LLMs such as ChatGPT, PaLM, and Claude, there may be even greater differences, but the implemented function and effects are usually similar.

8. Conclusions

Pre-training large models such as ChatGPT and GPT-4 have revolutionized software development by providing a new possibility for automated program code generation. We focus on the demand for automatic code generation in safety-critical domains such as nuclear energy and automotive industries. We use real-world engineering cases to conduct a case study on generating safety-critical software code using GPT-4 as the code generation tool. By exploring different approaches and methods, a novel Prompt engineering (Prompt-FDC) suitable for generating code for safety-critical software is introduced, and a new software development process and lifecycle V-model for safety software are proposed. Through the practical experiments described in this paper, we demonstrate that, with appropriate prompt engineering, LLMs can auto-generate safety-critical software code that can be applied in practical engineering. The LLMs are still evolving, aiming to improve the understanding of requirements and increase code quality. Applying LLMs-based code generation techniques to more engineering domains to address efficiency and safety issues in software development is an area that deserves attention and further research in the future.

There are several directions for future work. The code generation accomplished in this paper mainly focuses on functional components in safety-critical software. Researching code generation on a larger scale would bring greater convenience to industrial applications. Another important aspect to consider is the efficient and accurate verification of code generated by LLMs. So far, software verification relies mainly on manual code review and testing. It is crucial to explore verification methods that are more suitable for automatically generated code. Additionally, to facilitate the refinement and enhancement of safety-critical software requirements, prompt and code plugins based on GPT-4 can be developed to better support new software development processes.

Author Contributions: Conceptualization, M.L. and J.W.; methodology, Q.M.; software, Z.F.; validation, Y.W. and T.L.; formal analysis, M.L.; investigation, Y.W.; resources, J.W. and Q.M.; data curation, Z.F.; writing—original draft preparation, M.L.; writing—review and editing, J.W.; visualization, Y.W.; supervision, T.L.; project administration, Q.M.; funding acquisition, J.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was funded by the National Key Research and Development Program under Grant 2022YFB3305200; and in part by the Major Science and Technology Special Project of Sichuan Province under Grant 2022ZDZX0008.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The authors declare that the data supporting the findings of this study are available at the following URL: <https://github.com/lmxstar/GPT-prompts-for-safety-critical-software> (accessed on 25 December 2023).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Generated Code for NucRate Algorithm

- (1) Generated code based on the NucRate algorithm's overall requirements is shown in Listing A1:

Listing A1. Generated code based on NucRate algorithm's overall requirements.

```

1.  #include <stdio.h>
2.
3.  #define CHANNELS 4
4.  #define THRESHOLD 10
5.
6.  typedef struct {
7.      int powerSignal;
8.      int resetSignal;
9.  } RSFlipFlop;
10.
11. typedef struct {
12.     int qualityBit;
13.     int stopSignal;
14. } VoterInput;
15.
16. VoterInput voterInputs[CHANNELS];
17. RSFlipFlop rsFlipFlop;
18.
19. void processPowerSignal(int channel, int signal) {
20.     if (signal > THRESHOLD) {
21.         rsFlipFlop.powerSignal = 1;
22.     }
23.     voterInputs[channel].stopSignal = rsFlipFlop.powerSignal;
24. }
25.
26. void processResetSignal(int signal) {
27.     rsFlipFlop.resetSignal = signal;
28. }
29.
30. void processVoterInputs() {
31.     int goodInputs = 0;
32.     int badInputs = 0;
33.     for (int i = 0; i < CHANNELS; i++) {
34.         if (voterInputs[i].qualityBit == 1) {
35.             goodInputs++;
36.         } else {
37.             badInputs++;
38.         }
39.     }
40.
41.     if (goodInputs >= 2) {
42.         // Output two stop signals
43.         printf("Output two stop signals\n");
44.     } else if (goodInputs == 1) {
45.         // Output one stop signal
46.         printf("Output one stop signal\n");
47.     } else {
48.         // All signals are bad, output action signal
49.         printf("Output action signal\n");
50.     }
51. }
52.
53. int main() {
54.     // Initialize RS flip flop
55.     rsFlipFlop.powerSignal = 0;
56.     rsFlipFlop.resetSignal = 0;
57.
58.     // Initialize voter inputs
59.     for (int i = 0; i < CHANNELS; i++) {
60.         voterInputs[i].qualityBit = 1;
61.         voterInputs[i].stopSignal = 0;
62.     }
63.
64.     // Process signals
65.     for (int i = 0; i < CHANNELS; i++) {
66.         processPowerSignal(i, i * 3); // Example power signals
67.     }
68.     processResetSignal(1); // Example reset signal
69.
70.     // Process voter inputs
71.     processVoterInputs();
72.
73.     return 0;
74. }
75.

```

- (2) Generated code based on the NucRate algorithm's specific requirements is shown in Listing A2:

Listing A2. Generated code based on NucRate algorithm's specific requirements.

```

1.  #include <stdbool.h>
2.
3.  // Input signal names and sources
4.  float RNI1, RNI2, RNI3, RNI4; // Neutron flux rate positive change signals from 4 measuring
   channels
5.  bool RNI1_Q, RNI2_Q, RNI3_Q, RNI4_Q; // Quality bits for each signal
6.  bool RNI_RST; // External reset signal
7.
8.  // Output signal names
9.  bool RT1; // Shutdown signal sent to dedicated device A
10. bool RT2; // Shutdown signal sent to shutdown circuit breaker
11.
12. // Function prototypes
13. void update_quality_bits(void);
14. bool hysteresis_comparator(float signal, float high_threshold, float low_threshold);
15. void RS_flip_flops(bool *RS_outputs, bool S_input, bool R_input);
16. void majority_vote(bool *output1, bool *output2, bool *inputs, bool *quality_bits);
17.
18. int main() {
19.     bool RS_outputs [4] = {false, false, false, false};
20.     bool majority_vote_inputs[4];
21.
22.     update_quality_bits();
23.
24.     for(int i = 0; i < 4; i++) {
25.         float RNI_signal = (i == 0) ? RNI1 : (i == 1) ? RNI2 : (i == 2) ? RNI3 : RNI4;
26.         majority_vote_inputs[i] = hysteresis_comparator(RNI_signal, 100, 95);
27.     }
28.
29.     for(int i = 0; i < 4; i++) {
30.         RS_flip_flops(&RS_outputs[i], majority_vote_inputs[i], RNI_RST);
31.     }
32.
33.     majority_vote(&RT1, &RT2, RS_outputs, (bool[]){RNI1_Q, RNI2_Q, RNI3_Q, RNI4_Q});
34.
35.     return 0;
36. }
37.
38. // Update the quality bits for each signal based on their range
39. void update_quality_bits() {
40.     float RNI_signals[] = {RNI1, RNI2, RNI3, RNI4};
41.     bool *quality_bits[] = {&RNI1_Q, &RNI2_Q, &RNI3_Q, &RNI4_Q};
42.
43.     for(int i = 0; i < 4; i++) {
44.         if (RNI_signals[i] < 10 || RNI_signals[i] > 10000) {
45.             *quality_bits[i] = false;
46.         }
47.     }
48. }
49.
50. // Hysteresis comparator function
51. bool hysteresis_comparator(float signal, float high_threshold, float low_threshold) {
52.     static bool output = false;
53.
54.     if (signal >= high_threshold) {
55.         output = true;
56.     } else if (signal <= low_threshold) {
57.         output = false;
58.     }
59.
60.     return output;
61. }
62.
63. // RS flip-flop function
64. void RS_flip_flops(bool *RS_output, bool S_input, bool R_input) {
65.     if (S_input) {
66.         *RS_output = true;
67.     } else if (R_input) {
68.         *RS_output = false;
69.     }
70. }
71.
72. // Majority vote function with degraded functionality depending on quality bits
73. void majority_vote(bool *output1, bool *output2, bool *inputs, bool *quality_bits) {
74.     int valid_count = 0;
75.     int threshold = 2;
76.
77.     for (int i = 0; i < 4; i++) {
78.         if (quality_bits[i]) {
79.             valid_count++;
80.         }
81.     }

```

```

82.
83.     if (valid_count < 2) {
84.         threshold = 1;
85.     }
86.
87.     int active_inputs = 0;
88.     for (int i = 0; i < 4; i++) {
89.         if (inputs[i] && quality_bits[i]) {
90.             active_inputs++;
91.         }
92.     }
93.
94.     if (active_inputs >= threshold) {
95.         *output1 = true;
96.         *output2 = true;
97.     } else {
98.         *output1 = false;
99.         *output2 = false;
100.    }
101. }
102.

```

- (3) Generated code based on software requirements through augmented prompt is shown in Listing A3:

Listing A3. Generated code based on NucRate algorithm's requirements through augmented prompt.

```

1. #include <stdbool.h> // For boolean type
2. #include <stdint.h> // For uint8_t type
3.
4. // Constants for input range validation
5. #define UPPER_LIMIT (10000.0f) // Upper limit for input signal
6. #define LOWER_LIMIT (10.0f) // Lower limit for input signal
7.
8. // Hysteresis thresholds
9. #define HYSTERESIS_HIGH_THRESHOLD (100.0f) // High threshold for hysteresis
10. #define HYSTERESIS_LOW_THRESHOLD (95.0f) // Low threshold for hysteresis
11.
12. // Function prototypes
13. static bool hysteresisThresholdComparator(float input, float *previous_output);
14. static void rsFlipFlop(bool set, bool reset, bool *output);
15. static bool fourOutOfTwoVoter(bool inputs[], bool quality_bits[]);
16. static void checkSignalRange(float *value, bool *quality);
17.
18. // Main processing function that evaluates neutron flux signals
19. void processNeutronFluxSignals(float inputs[], bool quality_bits[], bool reset, bool *RT1,
    bool *RT2) {
20.     static float previous_outputs[4] = {0.0f}; // Store previous outputs for hysteresis or
21.     bool flip_flop_outputs[4] = {false}; // Store outputs from RS flip-flop logic
22.
23.     // Check each signal's range and update quality bits accordingly
24.     for (uint8_t i = 0; i < 4; ++i) {
25.         checkSignalRange(&inputs[i], &quality_bits[i]);
26.     }
27.
28.     // Apply hysteresis comparator and RS flip-flop logic to each signal
29.     for (uint8_t i = 0; i < 4; ++i) {
30.         bool action_signal = hysteresisThresholdComparator(inputs[i],
            &previous_outputs[i]);
31.         rsFlipFlop(action_signal, reset, &flip_flop_outputs[i]);
32.     }
33.
34.     // Use voter logic to determine the final shutdown signals
35.     *RT1 = fourOutOfTwoVoter(flip_flop_outputs, quality_bits);
36.     *RT2 = *RT1; // Duplicate the first signal for the second shutdown mechanism
37. }
38.
39. // Function implementations

```

```

40. // Comparator that applies hysteresis to an input signal
41. static bool hysteresisThresholdComparator(float input, float *previous_output) {
42.     // If input is above the high threshold, signal is on
43.     if (input > HYSTERESIS_HIGH_THRESHOLD) {
44.         *previous_output = input;
45.         return true;
46.     }
47.     // If input drops below low threshold and was previously above it, signal is off
48.     if (input < HYSTERESIS_LOW_THRESHOLD && *previous_output > HYSTERESIS_LOW_THRESHOLD) {
49.         *previous_output = input;
50.         return false;
51.     }
52.     // Return the last known state if thresholds are not crossed
53.     return *previous_output > HYSTERESIS_LOW_THRESHOLD;
54. }
55.
56. // RS flip-flop logic to maintain or reset the state based on input signals
57. static void rsFlipFlop(bool set, bool reset, bool *output) {
58.     if (set) {
59.         *output = true; // Set output to true if set signal is received
60.     } else if (reset) {
61.         *output = false; // Reset output to false if reset signal is received
62.     }
63.     // If neither set nor reset signals are received, output remains unchanged
64. }
65.
66. // Voter function that decides the shutdown signal based on input and quality
67. static bool fourOutOfTwoVoter(bool inputs[], bool quality_bits[]) {
68.     uint8_t valid_inputs = 0;
69.     uint8_t bad_quality_count = 0;
70.
71.     // Count valid inputs and bad quality signals
72.     for (uint8_t i = 0; i < 4; ++i) {
73.         if (inputs[i]) {
74.             valid_inputs++;
75.         }
76.         if (!quality_bits[i]) {
77.             bad_quality_count++;
78.         }
79.     }
80.
81.     // Decision logic based on the counts
82.     switch (bad_quality_count) {
83.         case 0:
84.             case 1:
85.                 // If at most one bad quality signal, at least two valid inputs trigger shutdown
86.                 return valid_inputs >= 2;
87.             case 2:
88.             case 3:
89.                 // If two or three bad quality signals, at least one valid input triggers shutdown
90.                 return valid_inputs >= 1;
91.             default:
92.                 // If all signals are of bad quality, trigger shutdown
93.                 return true;
94.     }
95. }
96.
97. // Check if the signal is within the acceptable range and update quality bit
98. static void checkSignalRange(float *value, bool *quality) {
99.     // Signal is out of range if it's below the lower limit or above the upper limit
100.    if (*value < LOWER_LIMIT || *value > UPPER_LIMIT) {
101.        *quality = false; // Set quality bit to false for out of range signal
102.    } else {
103.        *quality = true; // Set quality bit to true for signal within the range
104.    }
105.}

```

Appendix B. Generated Code for CruiseControl Software

- (1) Generated code based on CruiseControl's overall requirements is shown in Listing A4:

Listing A4. Generated code based on CruiseControl’s overall requirements.

```

1.  #include <stdio.h>
2.
3.  // CC parameters
4.  #define MIN_SPEED 20
5.  #define MAX_SPEED 120
6.  #define SPEED_INCREMENT 5
7.  #define MAX_THROTTLE_SATURATION 100
8.  #define MIN_PEDAL_PRESS_DETECTION 10
9.
10. // CC states
11. enum CCState { OFF, ON, STDBY };
12.
13. // CC variables
14. int cruiseSpeed = 0;
15. int carSpeed = 0;
16. int throttleCommand = 0;
17. enum CCState ccState = OFF;
18. int acceleratorPedal = 0;
19. int brakePedal = 0;
20.
21. // Function to update CC state
22. void updateCCState() {
23.     if (ccState == OFF) {
24.         if (carSpeed >= MIN_SPEED && carSpeed <= MAX_SPEED && acceleratorPedal <
25.             MIN_PEDAL_PRESS_DETECTION) {
26.             ccState = ON;
27.             cruiseSpeed = carSpeed;
28.         }
29.     } else if (ccState == ON) {
30.         if (acceleratorPedal >= MIN_PEDAL_PRESS_DETECTION || carSpeed < MIN_SPEED ||
31.             carSpeed > MAX_SPEED) {
32.             ccState = STDBY;
33.         } else if (brakePedal >= MIN_PEDAL_PRESS_DETECTION) {
34.             ccState = OFF;
35.         }
36.     } else if (ccState == STDBY) {
37.         if (acceleratorPedal < MIN_PEDAL_PRESS_DETECTION && carSpeed >= MIN_SPEED &&
38.             carSpeed <= MAX_SPEED) {
39.             ccState = ON;
40.         }
41.     }
42. }
43. // Function to update throttle command
44. void updateThrottleCommand() {
45.     if (ccState == ON) {
46.         throttleCommand = (cruiseSpeed - carSpeed) * MAX_THROTTLE_SATURATION / (MAX_SPEED
47.             - MIN_SPEED);
48.         if (throttleCommand > MAX_THROTTLE_SATURATION) {
49.             throttleCommand = MAX_THROTTLE_SATURATION;
50.         }
51.     } else {
52.         throttleCommand = 0;
53.     }
54. }
55. // Main function
56. int main() {
57.     // Simulate car and driver actions
58.     carSpeed = 50;
59.     acceleratorPedal = 0;
60.     brakePedal = 0;
61.     updateCCState();
62.     updateThrottleCommand();
63.     printf("CC State: %d, Cruise Speed: %d, Throttle Command: %d\n", ccState, cruiseSpeed,
64.         throttleCommand);
65.     return 0;
66. }

```

- (2) Generated code based on CruiseControl’s specific requirements is shown in Listing A5:

Listing A5. Generated code based on CruiseControl's specific requirements.

```

1.  #include <stdbool.h>
2.  #include <stddef.h>
3.
4.  // Input signal names and sources
5.  typedef enum {
6.      CC_OFF,
7.      CC_ON,
8.      CC_RESUME,
9.      CC_SET,
10.     CC_QUICK_DECEL,
11.     CC_QUICK_ACCEL
12. } CC_Commands;
13.
14. typedef struct {
15.     CC_Commands command;
16.     float speed;
17.     float accel;
18.     float brake;
19. } CC_Input;
20.
21. // Output signal names
22. typedef struct {
23.     float cruise_speed;
24.     bool throttle_cmd;
25.     enum { CS_OFF, CS_ON, CS_STDBY, CS_INT } cruise_state;
26. } CC_Output;
27.
28. // Function prototypes
29. void cruise_control(CC_Input input, CC_Output *output);
30. void cruise_speed_management(CC_Input input, CC_Output *output);
31. void car_driving_control(CC_Input input, CC_Output *output);
32. void pedals_pressed_detection(CC_Input input, CC_Output *output);
33.
34. int main() {
35.     CC_Input input;
36.     CC_Output output;
37.
38.     // Update input signals here
39.
40.     cruise_control(input, &output);
41.
42.     return 0;
43. }
44.
45. // Main cruise control function
46. void cruise_control(CC_Input input, CC_Output *output) {
47.     cruise_speed_management(input, output);
48.     car_driving_control(input, output);
49.     pedals_pressed_detection(input, output);
50. }
51.
52. // Cruise speed management functionality
53. void cruise_speed_management(CC_Input input, CC_Output *output) {
54.     const float SpeedMin = 30.0;
55.     const float SpeedMax = 150.0;
56.     const float SpeedInc = 2.5;
57.
58.     if (output->cruise_state == CS_ON || output->cruise_state == CS_STDBY ||
59.         output->cruise_state == CS_INT) {
60.         if (input.command == CC_SET) {
61.             output->cruise_speed = input.speed;
62.         } else if (input.command == CC_QUICK_ACCEL) {
63.             output->cruise_speed = (output->cruise_speed + SpeedInc <= SpeedMax) ?
64.                 output->cruise_speed + SpeedInc : SpeedMax;
65.         } else if (input.command == CC_QUICK_DECEL) {
66.             output->cruise_speed = (output->cruise_speed - SpeedInc >= SpeedMin) ?
67.                 output->cruise_speed - SpeedInc : SpeedMin;
68.         }
69.     }
70. }

```

```

68. // Car driving control functionality
69. void car_driving_control(CC_Input input, CC_Output *output) {
70.     const float Kp = 8.113;
71.     const float Ki = 0.5;
72.     const float ThrottleSatMax = 45.0;
73.
74.     // Implement the proportional and integral algorithm here, using Kp and Ki factors
75.
76.     // Limit the throttle command to ThrottleSatMax to ensure comfort during
    //automatic regulation
77.     if (output->throttle_cmd > ThrottleSatMax) {
78.         output->throttle_cmd = ThrottleSatMax;
79.     }
80. }
81.
82. // Pedals pressed detection functionality
83. void pedals_pressed_detection(CC_Input input, CC_Output *output) {
84.     const float PedalsMin = 3.0;
85.
86.     bool accel_pressed = input.accel > PedalsMin;
87.     bool brake_pressed = input.brake > PedalsMin;
88.
89.     // Update CruiseState based on pedal presses and CC commands
90.     switch (input.command) {
91.         case CC_OFF:
92.             output->cruise_state = CS_OFF;
93.             break;
94.         case CC_ON:
95.             output->cruise_state = CS_ON;
96.             break;
97.         case CC_RESUME:
98.             if (!accel_pressed && (input.speed >= 30.0 && input.speed <= 150.0)) {
99.                 output->cruise_state = CS_ON;
100.            } else {
101.                output->cruise_state = CS_STDBY;
102.            }
103.            break;
104.            default:
105.                break;
106.        }
107.
108.        if (brake_pressed) {
109.            output->cruise_state = CS_INT;
110.        } else if (accel_pressed || input.speed < 30.0 || input.speed > 150.0) {
111.            output->cruise_state = CS_STDBY;
112.        } else if (!accel_pressed && (input.speed >= 30.0 && input.speed <= 150.0)) {
113.            output->cruise_state = CS_ON;
114.        }
115.    }

```

- (3) Generated code based on software requirements through augmented prompt is shown in Listing A6:

Listing A6. Generated code based on CruiseControl's requirements through augmented prompt.

```

1.  #include <stdbool.h>
2.  #include <stdint.h>
3.
4.  // Constants for cruise control system
5.  #define SPEED_MIN 30.0f // Minimum cruise speed
6.  #define SPEED_MAX 150.0f // Maximum cruise speed
7.  #define SPEED_INC 2.5f // Speed increment/decrement for quick accel/decel
8.  #define KP 8.113f // Proportional gain
9.  #define KI 0.5f // Integral gain
10. #define THROTTLE_SAT_MAX 45.0f // Maximum throttle saturation value
11. #define PEDALS_MIN 3.0f // Minimum pedal value to detect pedal press
12.
13. // CruiseState enum for different cruise control states
14. typedef enum
15. {
16.     CRUISE_OFF, // Cruise control is off
17.     CRUISE_ON, // Cruise control is on and active
18.     CRUISE_STDBY, // Cruise control is on standby
19.     CRUISE_INT // Cruise control is interrupted by brake pedal
20. } CruiseState;
21.
22. // Car driving control function calculates throttle command based on speed error and //
integral
23. static float car_driving_control(float speed_error, float *integral, bool is_on,
bool is_saturated)
24. {
25.     if (!is_on)
26.     {
27.         // Reset integral when cruise control is off
28.         *integral = 0.0f;
29.         return 0;
30.     }
31.
32.     if (!is_saturated)
33.     {
34.         // Update integral if throttle is not saturated
35.         *integral += speed_error;
36.     }
37.
38.     // Calculate control output using proportional and integral gains
39.     float output = (KP * speed_error) + (KI * (*integral));
40.     return output;
41. }
42.
43. // Cruise speed management function updates cruise speed based on user inputs
44. static float cruise_speed_management(float current_speed, CruiseState state,
bool set, bool quick_accel, bool quick_decel)
45. {
46.     static float cruise_speed = 0.0f; // Maintain the current cruise speed
47.
48.     if ((state != CRUISE_ON) && (state != CRUISE_STDBY) && (state != CRUISE_INT))
49.     {
50.         return cruise_speed;
51.     }
52.     // Update cruise speed according to user inputs
53.     if (set)
54.     {
55.         cruise_speed = current_speed; // Set cruise speed to current speed
56.     }
57.     else if (quick_accel && (cruise_speed + SPEED_INC) <= SPEED_MAX)
58.     {
59.         cruise_speed += SPEED_INC; // Increase cruise speed
60.     }
61.     else if (quick_decel && (cruise_speed - SPEED_INC) >= SPEED_MIN)
62.     {
63.         cruise_speed -= SPEED_INC; // Decrease cruise speed
64.     }
65.
66.     return cruise_speed;
67. }
68.
69. // Pedals pressed detection function checks if a pedal is pressed based on its value
70. static bool is_pedal_pressed(float pedal_value)
71. {
72.     return pedal_value > PEDALS_MIN; // Returns true if pedal value is greater than // the
minimum threshold
73. }
74.
75. // Cruise control behavior function determines the new cruise control state based on //
provided inputs
76. static CruiseState cruise_control_behavior(CruiseState current_state, bool on,
bool off, bool resume, bool brake_pressed, bool accel_pressed, float speed)
77. {
78.     CruiseState new_state = current_state; // Initialize new_state with current_state

```

```

79.
80.    // Update cruise control state based on inputs
81.    if (off)
82.    {
83.        new_state = CRUISE_OFF;
84.    }
85.    else if (on)
86.    {
87.        new_state = CRUISE_ON;
88.    }
89.    else if (current_state == CRUISE_ON)
90.    {
91.        if (accel_pressed || (speed < SPEED_MIN) || (speed > SPEED_MAX))
92.        {
93.            new_state = CRUISE_STDBY;
94.        }
95.    }
96.
97.    else if ((current_state == CRUISE_STDBY) && (!accel_pressed) &&
98.            (speed >= SPEED_MIN) && (speed <= SPEED_MAX))
99.    {
100.        new_state = CRUISE_ON;
101.    }
102.    else if (brake_pressed)
103.    {
104.        new_state = CRUISE_INT;
105.    }
106.    else if (current_state == CRUISE_INT && resume)
107.    {
108.        if (!accel_pressed && (speed >= SPEED_MIN) && (speed <= SPEED_MAX))
109.        {
110.            new_state = CRUISE_ON;
111.        }
112.        else
113.        {
114.            new_state = CRUISE_STDBY;
115.        }
116.    }
117.    return new_state;
118. }
119.
120. // Top-Level function for Cruise Control
121. void cruise_control(const bool CC_INR_01, const bool CC_INR_02, const bool CC_INR_03 ,
122. const bool CC_INR_04, const bool CC_INR_05, const float CC_INR_06,
123. const float CC_INR_07, const float CC_INR_08, float *CC_OUTR_01,
124. bool *CC_OUTR_02, CruiseState *CC_OUTR_03)
125. {
126.    // Manage cruise control behavior based on inputs
127.    CruiseState new_state = cruise_control_behavior(*CC_OUTR_03, CC_INR_01, !CC_INR_01 ,
128. CC_INR_02, is_pedal_pressed(CC_INR_07), is_pedal_pressed(CC_INR_06), CC_INR_08);
129.
130.    // Manage cruise speed based on inputs and current cruise control state
131.    float cruise_speed = cruise_speed_management(CC_INR_08, new_state, CC_INR_03,
132. CC_INR_05, CC_INR_04);
133.
134.    // Calculate speed error between the current speed and desired cruise speed
135.    float speed_error = cruise_speed - CC_INR_08;
136.
137.    // Determine if throttle is saturated (reached maximum limit)
138.    bool is_saturated = *CC_OUTR_02 >= THROTTLE_SAT_MAX;
139.
140.    // Calculate throttle command based on speed error, integral, cruise control
141.    //state, and saturation
142.    static float integral = 0.0f;
143.    float throttle_cmd = car_driving_control(speed_error, &integral,
144. new_state == CRUISE_ON, is_saturated);
145.
146.    // Update output signals with calculated values
147.    *CC_OUTR_01 = cruise_speed; // Output cruise speed
148.    *CC_OUTR_02 = throttle_cmd; // Output throttle command
149.    *CC_OUTR_03 = new_state; // Output updated cruise control state
150. }

```

Appendix C. The Test Code for Robustness Testing

Listing A7. The test code for robustness testing.

```

1.  #include <stdio.h>
2.  #include <stdbool.h>
3.  #include <stdlib.h>
4.  #include <time.h>
5.  #include <assert.h>
6.
7.  // Function to generate a random float within a specified range
8.  float generateRandomFloat(float min, float max) {
9.      float random = ((float)rand()) / (float)RAND_MAX;
10.     return min + (random * (max - min));
11. }
12.
13. // Test case for boundary value testing
14. void boundaryValueTesting() {
15.     float inputs[4] = {LOWER_LIMIT - 1.0f, LOWER_LIMIT, UPPER_LIMIT, UPPER_LIMIT
16. + 1.0f};
17.     bool quality_bits[4] = {true, true, true, true};
18.     bool reset = false;
19.     bool RT1, RT2;
20.
21.     processNeutronFluxSignals(inputs, quality_bits, reset, &RT1, &RT2);
22.
23.     // Verify the expected outputs
24.     assert(RT1 == true);
25.     assert(RT2 == true);
26. }
27.
28. // Test case for exception input testing
29. void exceptionInputTesting() {
30.     float inputs[4] = {15.0f, -20.0f, 100.0f, atof("invalid")};
31.     bool quality_bits[4] = {true, true, true, true};
32.     bool reset = false;
33.     bool RT1, RT2;
34.
35.     processNeutronFluxSignals(inputs, quality_bits, reset, &RT1, &RT2);
36.
37.     // Verify the expected outputs
38.     assert(RT1 == true);
39.     assert(RT2 == true);
40. }
41.
42. // Test case for random input testing
43. void randomInputTesting() {
44.     srand(time(NULL));
45.     float inputs[4];
46.     bool quality_bits[4];
47.     bool reset = false;
48.     bool RT1, RT2;
49.
50.     for (int i = 0; i < 4; i++) {
51.         inputs[i] = generateRandomFloat(LOWER_LIMIT, UPPER_LIMIT);
52.         quality_bits[i] = true;
53.     }
54.
55.     processNeutronFluxSignals(inputs, quality_bits, reset, &RT1, &RT2);
56.
57.     // Verify the expected outputs
58.     // Add assertions or other result checks here
59. }
60.
61. // Test case for forced error testing
62. void forcedErrorTesting() {
63.     float inputs[4] = {90.0f, 92.0f, 94.0f, 96.0f};
64.     bool quality_bits[4] = {true, true, true, true};
65.     bool reset = true;
66.     bool RT1, RT2;
67.
68.     processNeutronFluxSignals(inputs, quality_bits, reset, &RT1, &RT2);
69.
70.     // Verify the expected outputs
71.     assert(RT1 == false);
72.     assert(RT2 == false);
73. }
74.
75. // Test case for security testing
76. void securityTesting() {
77.     float inputs[4] = {20000.0f, -5000.0f, 15000.0f, 30000.0f};
78.     bool quality_bits[4] = {true, false, true, true};
79.     bool reset = false;
80.     bool RT1, RT2;
81.
82.     processNeutronFluxSignals(inputs, quality_bits, reset, &RT1, &RT2);
83.
84.     // Verify the expected outputs

```

```

85.     assert(RT1 == true);
86.     assert(RT2 == true);
87. }
88.
89. int main() {
90.     boundaryValueTesting();
91.     exceptionInputTesting();
92.     randomInputTesting();
93.     forcedErrorTesting();
94.     securityTesting();
95.
96.     printf("All tests passed successfully!\n");
97.
98.     return 0;
99. }

```

Appendix D. The Standardized XML for Generalized Requirements

(1) The standardized XML template for generalized requirements is shown in Listing A8.

Listing A8. The standardized XML template for generalized requirements.

```

<requirements>
<input_signals>
<!-- Input signals -->
<signal>
<req_id></req_id>
<source></source>
</signal>
</input_signals>
<input_signal_types>
<!-- Input signal types -->
<signal_type>
<req_id></req_id>
<type></type>
<channels>
<channel></channel>
</channels>
</signal_type>
</input_signal_types>
<input_signal_ranges>
<!-- Input signal ranges -->
<signal_range>
<req_id></req_id>
<limits>
<lower_limit></lower_limit>
<upper_limit></upper_limit>
</limits>
<channels>
<channel></channel>
</channels>
</signal_range>
</input_signal_ranges>
<output_signals>
<!-- Output signals -->
<signal>
<req_id></req_id>
<destination></destination>
</signal>
</output_signals>
<output_signal_types>
<!-- Output signal types -->
<signal_type>
<req_id></req_id>
<type></type>
</signal_type>
</output_signal_types>
<output_signal_ranges>
<!-- output signal ranges -->
<signal_range>
<req_id></req_id>

```



```

<limits>
  <lower_limit></lower_limit>
  <upper_limit></upper_limit>
</limits>
<channels>
  <channel></channel>
</channels>
</signal_range>
</output_signal_ranges>
<program_parameters>
  <!-- Program parameters -->
</program_parameters>
<functional_requirements>
  <!-- Functional requirements -->
  <requirement>
    <req_id></req_id>
    <description></description>
  </requirement>
</functional_requirements>
<generalized_domain_requirements>
  <!-- Generalized domain requirements -->
  <requirement>
    <req_id></req_id>
    <description></description>
  </requirement>
</generalized_domain_requirements>
<constraints_requirements>
  <!-- Constraint requirements -->
  <requirement>
    <req_id></req_id>
    <description></description>
  </requirement>
</constraints_requirements>
</requirements>

```

- (2) The standardized XML generalized requirements of the NucRate algorithm are shown in Listing A9.

Listing A9. The standardized XML generalized requirements of the NucRate algorithm.

```

<requirements>
  <input_signals>
    <signal>
      <req_id>RNI_INR_01</req_id>
      <source>Positive rate of change of neutron flux from four measurement channels, RNI1, RNI2,
      RNI3, RNI4.</source>
    </signal>
    <signal>
      <req_id>RNI_INR_03</req_id>
      <source>External reset signal, RNI_RST.</source>
    </signal>
  </input_signals>
  <input_signal_types>
    <signal_type>
      <req_id>RNI_INR_03</req_id>
      <type>Analog</type>
      <channels>
        <channel>RNI1</channel>
        <channel>RNI2</channel>
        <channel>RNI3</channel>
        <channel>RNI4</channel>
      </channels>
    </signal_type>
    <signal_type>
      <req_id>RNI_INR_05</req_id>
      <type>Digital</type>
      <channels>
        <channel>RNI_RST</channel>
      </channels>
    </signal_type>
  </input_signal_types>
  <input_signal_ranges>
    <signal_range>
      <req_id>RNI_INR_06</req_id>
      <limits>
        <lower_limit>10</lower_limit>
        <upper_limit>10000</upper_limit>
      </limits>
      <channels>
        <channel>RNI1</channel>
        <channel>RNI2</channel>
        <channel>RNI3</channel>
        <channel>RNI4</channel>
      </channels>
    </signal_range>
  </input_signal_ranges>
  <output_signals>
    <signal>
      <req_id>RNI_OUTR_01</req_id>
      <destination>Dedicated location A</destination>
    </signal>
    <signal>
      <req_id>RNI_OUTR_02</req_id>
      <destination>Shutdown circuit breaker</destination>
    </signal>
  </output_signals>

```

```

<output_signal_types>
<signal_type>
<req_id>RNI_OUTR_01</req_id>
<type>Digital</type>
</signal_type>
<signal_type>
<req_id>RNI_OUTR_02</req_id>
<type>Digital</type>
</signal_type>
</output_signal_types>
<output_signal_ranges>
<signal_range>
<!-- No output_signal_ranges -->
</signal_range>
</output_signal_ranges>
<program_parameters>
<!-- No program parameters -->
</program_parameters>
<functional_requirements>
<requirement>
<req_id>RNI_FUNR_01</req_id>
<description>High signals of positive rate of change of neutron flux from the four channels are
first checked against the signal range. Signals that exceed the range will have their corresponding
quality bits set to bad.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_02</req_id>
<description>High signals of positive rate of change of neutron flux from the four channels are
compared with a hysteresis threshold comparator to generate action signals.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_03</req_id>
<description>The hysteresis threshold comparator is a threshold comparator with hysteresis func-
tion. The hysteresis high threshold is 100, and the hysteresis low threshold is 95.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_04</req_id>
<description>The action signal is input to the S terminal of an RS flip-flop, and the external reset
signal is input to the R terminals of the four flip-flops.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_05</req_id>
<description>The RS flip-flop has S priority.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_06</req_id>
<description>The output signals of the four RS flip-flops are input to a 4-to-2 voter. After the vot-
ing process, two shutdown signals are output. One shutdown signal is output to the dedicated
location A, and the other shutdown signal is sent to the shutdown circuit breaker.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_07</req_id>
<description>The 4-to-2 voter triggers an action when at least two out of the four inputs are valid.
The voter has a degradation feature where, if one input's quality bit is bad, it degrades to a 3-to-2
voter, and then to a 2-to-1 voter. When all quality bits of the signals are bad, the voter outputs an
action signal.</description>
</functional_requirements>
<generalized_domain_requirements>
<requirement>

```

```

<req_id>RNI_FUNR_07_01</req_id>
<description>The 4-out-of-2 voting logic takes action when at least two out of the four inputs are
valid. However, the voting result is influenced by the quality bits of the inputs.</description>
</requirement>
<requirement>
<req_id>RNI_FUNR_07_02</req_id>
<description>The degradation of the voting quality bit function for the 4-out-of-2 voting logic re-
fers to the following: when one input has a bad quality bit, it degrades to a 3-out-of-2 voting logic;
when two inputs have bad quality bits, it degrades to a 2-out-of-1 voting logic; when three inputs
have bad quality bits, it degrades to a 1-out-of-1 voting logic; when all signals have bad quality
bits, the voting logic directly outputs the action signal.</description>
</requirement>
</generalized_domain_requirements>
<constraints_requirements>
<requirement>
<req_id>RNI_NFUNR_01</req_id>
<description>Programming language constraint: C99 standard language.</description>
</requirement>
<requirement>
<req_id>RNI_NFUNR_02</req_id>
<description>Code standard limitation: Compliance with MISRA-C:2004 standard.</description>
</requirement>
<requirement>
<req_id>RNI_NFUNR_03</req_id>
<description>Code comment rate: Greater than 20%.</description>
</requirement>
<requirement>
<req_id>RNI_NFUNR_04</req_id>
<description>Program structure requirement (readability): Implement the hysteresis threshold
comparator, 4-out-of-2 voting logic, and RS flip-flop as separate functions.</description>
</requirement>
<requirement>
<req_id>RNI_NFUNR_05</req_id>
<description>Safety and security requirement: Static memory allocation.</description>
</requirement>
<requirement>
<req_id>RNI_NFUNR_06</req_id>
<description>Safety and security requirement: No use of interrupts.</description>
</requirement>
<requirement>
<req_id>RNI_NFUNR_07</req_id>
<description>Performance requirements: Execution time less than 100us.</description>
</requirement>
</constraints_requirements>
</requirements>

```

Appendix E. The Detail Procedures for Conducting the Experiments

Listing A10. The detailed procedures for conducting the experiments.

- (1) Prepare software domain requirements.
- (2) Itemize software requirements and break them down into smaller granularity requirements.
- (3) Translate software requirements into standardized XML requirements documents.
- (4) Carefully analyze the functional requirements of the software and generalize content that contains domain-specific terminology. The transformed requirements are in common vocabulary.
- (5) Provide the transformed software requirements as prompts to the GPT-4 dialog box and make it automatically generate code.
- (6) Review the integrity of generated code and check that code generation for all requirements is complete.
- (7) Focus on checking the code of the functional part of the domain requirements. In case of missing or incorrect code, modify the conversion content of the domain requirements to make its description clearer. The modified requirements are then provided as prompts to GPT-4 to regenerate the code.
- (8) Repeat until you are satisfied with the generated code.
- (9) Functional testing of the generated code.
- (10) Domain compliance checking of the generated code.
- (11) Robustness testing of the generated code.
- (12) For the problems found in the test process, the problem is provided as a new prompt to GPT-4 to make targeted changes to the code.
- (13) Repeat until GPT-4 solves all test problems.

References

1. Leveson, N.G. *Engineering a Safer World: Systems Thinking Applied to Safety*; The MIT Press: Cambridge, MA, USA, 2016.
2. Jolak, R.; Ho-Quang, T.; Chaudron, M.R.V.; Schifferers, R.R.H. Model-Based Software Engineering: A Multiple-Case Study on Challenges and Development Efforts. In Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, Association for Computing Machinery, New York, NY, USA, 14–19 October 2018; pp. 213–223.
3. Colaco, J.-L.; Pagano, B.; Pouzet, M. SCADE 6: A Formal Language for Embedded Critical Software Development (Invited Paper). In Proceedings of the 2017 International Symposium on Theoretical Aspects of Software Engineering (TASE), Sophia Antipolis, France, 13–15 September 2017; pp. 1–11.
4. Le Sergent, T. SCADE: A Comprehensive Framework for Critical System and Software Engineering. In *SDL 2011: Integrating System and Software Modeling*; Ober, I., Ed.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2011; Volume 7083, pp. 2–3, ISBN 978-3-642-25263-1.
5. Osaiweran, A.; Schuts, M.; Hooman, J. Experiences with Incorporating Formal Techniques into Industrial Practice. *Empir. Softw. Eng.* **2014**, *19*, 1169–1194. [CrossRef]
6. Wei, J.; Tay, Y.; Bommasani, R.; Raffel, C.; Zoph, B.; Borgeaud, S.; Yogatama, D.; Bosma, M.; Zhou, D.; Metzler, D.; et al. Emergent Abilities of Large Language Models. *arXiv* **2022**, arXiv:2206.07682.
7. Bubeck, S.; Chandrasekaran, V.; Eldan, R.; Gehrke, J.; Horvitz, E.; Kamar, E.; Lee, P.; Lee, Y.T.; Li, Y.; Lundberg, S.; et al. Sparks of Artificial General Intelligence: Early Experiments with GPT-4. *arXiv* **2023**, arXiv:2303.12712.
8. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Pinto, H.P.O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374.
9. Liu, J.; Xia, C.S.; Wang, Y.; Zhang, L. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *arXiv* **2023**, arXiv:2305.01210.
10. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017; Curran Associates Inc.: Red Hook, NY, USA, 2017; pp. 6000–6010.
11. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Retraining of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North, Louis, MI, USA, 14–17 November 2019; Association for Computational Linguistics: Minneapolis, MN, USA, 2019; pp. 4171–4186.
12. Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I. Improving Language Understanding by Generative Pre-Training. Available online: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf (accessed on 25 December 2023).
13. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models Are Few-Shot Learners. In Proceedings of the 34th International Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 6–12 December 2020; Curran Associates Inc.: Red Hook, NY, USA, 2020; pp. 1877–1901.

14. Fedus, W.; Zoph, B.; Shazeer, N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *J. Mach. Learn. Res.* **2022**, *23*, 5232–5270.
15. Du, N.; Huang, Y.; Dai, A.M.; Tong, S.; Lepikhin, D.; Xu, Y.; Krikun, M.; Zhou, Y.; Yu, A.W.; Firat, O.; et al. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In Proceedings of the 39th International Conference on Machine Learning, Baltimore, MD, USA, 17–23 July 2022.
16. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training Language Models to Follow Instructions with Human Feedback. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 27730–27744.
17. Introducing ChatGPT. Available online: <https://openai.com/blog/chatgpt> (accessed on 25 December 2023).
18. Open, A.I.; Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; et al. GPT-4 Technical Report. *arXiv* **2023**, arXiv:2303.08774.
19. Anil, R.; Dai, A.M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. PaLM 2 Technical Report. *arXiv* **2023**, arXiv:2305.10403.
20. Anthropic\ Claude 2. Available online: <https://www.anthropic.com/index/claude-2> (accessed on 24 December 2023).
21. Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv* **2023**, arXiv:2307.09288.
22. Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Lago, A.D.; et al. Competition-Level Code Generation with AlphaCode. *Science* **2022**, *378*, 1092–1097. [[CrossRef](#)] [[PubMed](#)]
23. Nijkamp, E.; Pang, B.; Hayashi, H.; Tu, L.; Wang, H.; Zhou, Y.; Savarese, S.; Xiong, C. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *arXiv* **2022**, arXiv:2203.13474.
24. Wang, Y.; Le, H.; Gotmare, A.D.; Bui, N.D.Q.; Li, J.; Hoi, S.C.H. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. *arXiv* **2023**, arXiv:2305.07922.
25. Fried, D.; Aghajanyan, A.; Lin, J.; Wang, S.; Wallace, E.; Shi, F.; Zhong, R.; Yih, W.; Zettlemoyer, L.; Lewis, M. InCoder: A Generative Model for Code Infilling and Synthesis. *arXiv* **2022**, arXiv:2204.05999.
26. Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. Program Synthesis with Large Language Models. *arXiv* **2021**, arXiv:2108.07732.
27. Luo, Z.; Xu, C.; Zhao, P.; Sun, Q.; Geng, X.; Hu, W.; Tao, C.; Ma, J.; Lin, Q.; Jiang, D. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *arXiv* **2023**, arXiv:2306.08568.
28. Lai, Y.; Li, C.; Wang, Y.; Zhang, T.; Zhong, R.; Zettlemoyer, L.; Yih, S.W.; Fried, D.; Wang, S.; Yu, T. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In Proceedings of the 40th International Conference on Machine Learning, Honolulu, HI, USA, 23–29 July 2023.
29. White, J.; Fu, Q.; Hays, S.; Sandborn, M.; Olea, C.; Gilbert, H.; Elnashar, A.; Spencer-Smith, J.; Schmidt, D.C. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv* **2023**, arXiv:2302.11382.
30. Van Dis, E.A.M.; Bollen, J.; Zuidema, W.; Van Rooij, R.; Bockting, C.L. ChatGPT: Five Priorities for Research. *Nature* **2023**, *614*, 224–226. [[CrossRef](#)]
31. Jung, J.; Qin, L.; Welleck, S.; Brahman, F.; Bhagavatula, C.; Bras, R.L.; Choi, Y. Maieutic Prompting: Logically Consistent Reasoning with Recursive Explanations. *arXiv* **2022**, arXiv:2205.11822.
32. Arora, S.; Narayan, A.; Chen, M.F.; Orr, L.; Guha, N.; Bhatia, K.; Chami, I.; Re, C. Ask Me Anything: A Simple Strategy for Prompting Language Models. *arXiv* **2022**, arXiv:2210.02441.
33. Zhou, Y.; Muresanu, A.I.; Han, Z.; Paster, K.; Pitit, S.; Chan, H.; Ba, J. Large Language Models Are Human-Level Prompt Engineers. *arXiv* **2022**, arXiv:2211.01910.
34. Liu, V.; Chilton, L.B. Design Guidelines for Prompt Engineering Text-to-Image Generative Models. In Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, New Orleans, LA, USA, 29 April–5 May 2022.
35. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.
36. Koziol, H.; Gruener, S.; Ashiwal, V. ChatGPT for PLC/DCS Control Logic Generation. *arXiv* **2023**, arXiv:2305.15809.
37. IEC 61508:2010; Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems. International Electrotechnical Commission: Geneva, Switzerland, 2010.
38. IEC 60880:2006; Nuclear Power Plants—Instrumentation and Control Systems Important to Safety—Software Aspects for Computer-Based Systems Performing Category A Functions. International Electrotechnical Commission: Geneva, Switzerland, 2006.
39. ISO 26262:2011; Road Vehicles Functional Safety. International Organization for Standardization: Geneva, Switzerland, 2011.
40. Robustness Testing: What Is It & How to Deliver Reliable Software Systems with Test Automation. Available online: <https://www.parasoftchina.cn/what-is-robustness-testing/> (accessed on 9 January 2024).
41. Cppcheck—A Tool for Static C/C++ Code Analysis. Available online: <http://cppcheck.net/> (accessed on 24 December 2023).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.